



FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO FINAL

TÍTULO DEL DESAFÍO: Diseño e implementación de un sistema integrado de base de datos segura, automatizada e inteligente para la gestión del ciclo de vida de los datos de la cevichería “La Esquina”

Autores:

Florián Pérez Deivys Edilberto - N00485294 (100% participación)
Merino Zegarra Neda Victoria - N00492075 (100% participación)
Sáenz Véliz Amir Imanol - N00466501 (100% participación)
Salazar Medina Haziel Marco Eliel - N00466579 (100% participación)
Vásquez Marín Elmer Lizandro - N00499666 (100% participación)

Curso:

BASE DE DATOS AVANZADAS Y BIG DATA - CIIN1021P

Docente del Curso:

MARLON JOER LINAREZ VIGO

Cajamarca – Perú

2026-2

Contenido

I.	CAPÍTULO 1.....	3
	Objetivos del proyecto.....	3
	1.1. Relevamiento, supuestos y articulación curricular	4
	1.2. Automatización en SQL Server.....	8
	1.3. Implementación de políticas de seguridad y cumplimiento normativo.....	14
	1.4. Integración SQL–NoSQL	20
	1.5. Data Warehouse y procesos ETL	23
	1.6. Dashboard BI, KPIs y análisis OLAP	26
	1.7. Análisis Big Data con Apache Spark y evaluación de escalabilidad	31
	1.8. Reflexión ética y responsabilidad profesional.....	35
	1.9. Repositorio de código y trazabilidad del proyecto	37
	1.10. Planificación, cronograma y ruta crítica.....	39
	Conclusiones.....	41
	Referencias	42
II.	ANEXOS.....	43

I. CAPÍTULO 1.

Objetivos del proyecto

Objetivo general (OG). Proponer un sistema integrado de base de datos para la cevichería “La Esquina” que automatice los procesos de base de datos con T-SQL, sustente la toma de decisiones mediante inteligencia de negocios y gestione datos estructurados y semiestructurados con Big Data, garantizando seguridad y trazabilidad conforme a la Ley N.º 29733 (2011).

Objetivos específicos (OE). Cada uno de los cuatro objetivos tiene una meta medible y un plazo: el cierre del ciclo 2026-2, en la semana 8. La Tabla 1 relaciona cada meta con el resultado que medimos y con la sección donde está la evidencia.

Tabla 1

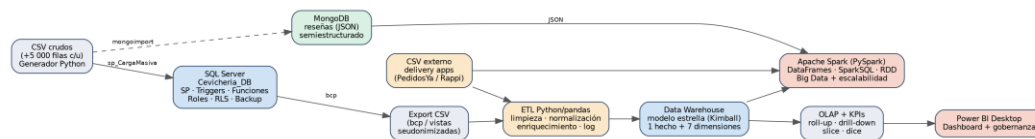
Objetivos específicos, indicadores SMART y resultados obtenidos

Obj	Objetivo específico	Indicador SMART (meta)	Resultado medido	Secc
OE 1	Automatizar los procesos de base de datos con T-SQL	≥ 2 procedimientos, ≥ 2 triggers y ≥ 1 función; 100 % de rechazos registrados en log	4 procedimientos, 5 triggers, 3 funciones; 11 982 pedidos cargados y 18 rechazados con registro en Log_Errores	1.2, 1.3
OE 2	Habilitar la toma de decisiones con inteligencia de negocios (DW, ETL, OLAP y dashboard)	≥ 1 hecho y ≥ 3 dimensiones; ≥ 3 KPIs; ≥ 1 operación OLAP; ETL en < 5 s	1 hecho y 7 dimensiones; 11 KPIs (5 en tarjetas); 5 operaciones OLAP; ETL en 1.52 s	1.5, 1.6
OE 3	Procesar Big Data estructurado y semiestructurado con Apache Spark	≥ 2 objetos Spark; ≥ 3 fuentes; comparación de tiempos en ≥ 3 volúmenes	3 objetos (RDD, DataFrame, SparkSQL); 4 fuentes (DW, CSV externo, JSON, asistencia); 3 volúmenes (hasta 2 043 350 filas)	1.7
OE 4	Integrar las soluciones en un flujo único, seguro y trazable	Pipeline SQL $\rightarrow$ CSV $\rightarrow$ ETL $\rightarrow$ DW $\rightarrow$ BI/Spark reproducible con README y matriz de trazabilidad	Repositorio con 8 carpetas, orden de ejecución y matriz de 8 filas	1.9

Nota. OG = objetivo general; OE = objetivo específico. Elaboración propia.

Objetivo de negocio (SMART). Al cierre del ciclo, la gerencia debe contar con un tablero de al menos cinco KPIs de ventas de los últimos 12 meses, que se actualice con un solo proceso ETL (menos de cinco segundos con el volumen actual) y le permita ver qué platos y canales dejan más ganancia. Los KPIs se definen y gobiernan en la sección 1.6.

Figura 1
Arquitectura de la solución integrada



Nota. Flujo de datos del proyecto: los CSV crudos se cargan y validan en SQL Server; los datos limpios se exportan a CSV, se integran con el CSV externo de plataformas de delivery mediante un ETL en pandas y alimentan el Data Warehouse (esquema en estrella). Del DW salen los KPIs, el análisis OLAP y el tablero de Power BI; Apache Spark consume el DW, el CSV externo y el JSON de reseñas. Elaboración propia con Graphviz.

1.1. Relevamiento, supuestos y articulación curricular

Trabajamos con la cevichería “La Esquina” de Cajamarca. Como no contamos con datos reales, usamos datos simulados con un generador reproducible en Python (generar_datos.py, semilla fija, ejecutado el 29 de septiembre de 2026). Ese generador conserva los 15 registros de personal generado con mockaroo y produce más de 5 000 filas en cada dataset transaccional. Los problemas de calidad se inyectaron de forma controlada para que su conteo se pueda verificar (Tabla 3).

1.1.1. Ficha de relevamiento de datos

La Tabla 2 resume los nueve conjuntos de datos del proyecto. Los maestros son pequeños, como es normal en un negocio de este tamaño (una carta de 40 platos y 25 trabajadores), en cambio, todos los datasets transaccionales, el CSV externo y el JSON de reseñas superan las 5 000 filas.

Tabla 2
Conjuntos de datos del proyecto

Dataset (archivo)	Tipo	Filas	Origen y uso
Cientes_Cevicheria.csv	Transaccional	5 545 (5 500 únicos)	Generador Python; carga a SQL Server
Pedidos_Cevicheria.csv	Transaccional	12 038 (12 000 únicos)	Generador Python (base en Mockaroo); carga a SQL Server
Detalle_Pedido_Cevicheria.csv	Transaccional	35 519	Generador Python; líneas de cada pedido
Asistencia_Personal.csv	Transaccional	6 000	Generador Python; carga a SQL Server y Spark
Ventas_Delivery_Apps_Externo.csv	Externo (CSV)	5 460 (5 400 únicos)	Exportación simulada de PedidosYa y Rappi; ETL y Spark
Resenas_Clientes.json	Semiestructurado	5 200	Campos variables y anidados; MongoDB y Spark
Personal_Cevicheria.csv	Maestro	25	15 registros de mockaroo y 10 nuevos (vendedores, delivery)
Platos_Cevicheria.csv	Maestro (carta)	40	Nueve categorías; precio y costo estimado
Feriados_Peru_2026.csv	Referencia	20	Enriquecimiento de Dim_Fecha

Nota. Periodo simulado de pedidos: 1 de octubre de 2025 al 27 de septiembre de 2026; canales delivery externos desde el 1 de enero de 2026. Elaboración propia.

1.1.2. Tabla de problemas de calidad

Tabla 3
Problemas de calidad detectados y registros afectados

Tipo de problema	Dataset y campo	Descripción	Registros
Valores nulos	Pedido.id_cliente	Clientes de paso sin identificar	1 080
	Cliente.email	Correo no registrado	165
	Personal.salario_mensual	Remuneración variable por propinas	4
	Externo.precio_brutto	Precio faltante en el CSV de las apps	12
	Externo.calificación	Pedido sin calificar (ausencia natural)	739
Registros duplicados	Pedido / Cliente / Externo	Filas idénticas (38 / 45 / 60)	143

Tipo de problema	Dataset y campo	Descripción	Registros
Inconsistencias de formato	Pedido.fecha_pedido	Texto “Ayer” (52) y “No se sabe” (26)	78
	Cliente.dni	DNI con 7 dígitos en lugar de 8	32
	Externo.plato	Mayúsculas, sin tildes o “cebiche” en vez de “ceviche”	2 755
Valores fuera de regla	Pedido.monto_total	Montos negativos (12) o iguales a cero (6)	18
Integridad referencial	Detalle_Pedido	Líneas cuyo pedido padre fue rechazado	52

Nota. Conteos obtenidos con `simular_carga_sql.py` y con el log del ETL; coinciden con las reglas R1 a R6 de los procedimientos de carga. El conteo de nulos de `id_cliente` se realizó después de descartar duplicados.

1.1.3. Requerimientos funcionales

Tabla 4

Requerimientos funcionales, criterios de éxito verificables y resultado

Cód.	Requerimiento	Criterio de éxito verificable	Resultado
RF1	Ingesta automatizada y validada de pedidos	100 % de los pedidos en la tabla final con fecha válida y monto > 0; 100 % de rechazos en Log_Errores	11 982 aceptados; 18 rechazados y registrados (100 %)
RF2	Auditoría de cambios sensibles	Cada UPDATE o DELETE de salario, cargo o monto genera una fila con valor anterior y nuevo; el log no admite UPDATE ni DELETE	Triggers T3, T4 y T5 (sección 1.2)
RF3	Acceso diferenciado por rol	6 roles; el analista no lee Cliente ni Empleado; el cliente solo ve sus pedidos	Roles, DDM y RLS (sección 1.3)
RF4	Data Warehouse integrado y conciliado	0 llaves sustitutas nulas en el hecho; 0 discrepancias entre SUM(detalle) y monto_total	40 867 filas de hechos; 0 nulos; 0 discrepancias
RF5	Tablero con KPIs y OLAP	≥ 3 KPIs con fuente, fórmula y responsable; ≥ 1 operación OLAP demostrable	11 KPIs gobernados; 5 operaciones OLAP
RF6	Análisis Big Data	Spark procesa ≥ 3 fuentes y compara tiempos con un motor SQL en ≥ 3 volúmenes	4 fuentes; 3 volúmenes

Nota. Elaboración propia a partir de las necesidades del negocio y del desafío del curso.

1.1.4. Declaración de supuestos

Tabla 5

Supuestos adoptados y su efecto en el diseño

Cód.	Supuesto	Justificación técnica y decisión de diseño
S1	Los pedidos sin id_cliente pertenecen a “clientes de paso”	Se registra el cliente genérico 9999 para no invalidar la llave foránea; en el DW se conserva como miembro especial y se excluye del cálculo de clientes recurrentes.
S2	Un pedido con fecha inválida ocurrió en la fecha de carga	La carga usa TRY_CONVERT (formato 103) y reemplaza el valor inválido por la fecha del sistema; se marca fecha_imputada = 1 para poder excluirlo de análisis de tendencia.
S3	Los empleados sin salario reciben remuneración variable por propinas	La columna salario_mensual admite NULL; los reportes usan ISNULL o COALESCE. El salario no viaja al DW.
S4	Las ventas de PedidosYa y Rappi no identifican cliente ni vendedor	Se crean los miembros especiales “Cliente App Delivery” (sk 2) y “Sin vendedor (plataforma externa)” para mantener llaves no nulas en el hecho (Kimball y Ross, 2013).
S5	El costo estimado de un plato equivale entre 28 % y 42 % de su precio base	Permite calcular margen bruto sin un módulo de recetas; el margen es una estimación y así se declara en la gobernanza de KPIs.

Nota. Elaboración propia.

1.1.5. Articulación curricular

Saberes del curso Base de Datos (ciclo 3) que se retoman:

(a) **Diseño relacional y restricciones (DDL):** el esquema se normaliza a tercera forma normal, se modela con diagrama entidad-relación (Chen, 1976) y se refuerza con restricciones CHECK y llaves foráneas.

(b) **Llaves foráneas y normalización:** se extienden hacia el modelo dimensional del Data Warehouse, donde las llaves naturales pasan a llaves sustitutas.

(c) **Consultas con JOIN y funciones de agregación:** se llevan a ROLLUP, CUBE y GROUPING SETS para el análisis OLAP y a SparkSQL.

Cursos posteriores:

Sistemas Integrados de Gestión: el proyecto desarrolla la competencia de preparar, integrar y documentar datos limpios (ETL en pandas, DW y Spark) que luego alimentan modelos predictivos, por ejemplo de demanda por plato y día.

Seguridad Integral de Sistemas: desarrolla la competencia de construir una capa de datos segura y reutilizable (procedimientos almacenados, roles, auditoría y repositorio versionado con README) que una aplicación o servicio puede consumir sin reprogramar las reglas de negocio.

1.2. Automatización en SQL Server

Toda la capa transaccional está en SQL Server, con un esquema normalizado y objetos T-SQL que llevan las reglas de negocio al motor (Medina Serrano, 2015). El modelo entidad-relación (Figura 2) tiene tres tablas maestras (Cliente, Empleado y Plato), tres transaccionales (Pedido, Detalle_Pedido y Asistencia) y dos de control (Log_Auditoria y Log_Errores). Cada pedido tiene N líneas en Detalle_Pedido, y eso permite calcular el margen por plato en el Data Warehouse.

Esta decisión parte de la división clásica del SQL. Según Camuña Rodríguez (2014), el DDL define la estructura, las restricciones y los objetos del modelo relacional, mientras que el DML se encarga de insertar, actualizar y consultar los datos, y así permite que las reglas del negocio vivan en el propio motor.

En nuestro caso, el DDL define las tablas, las restricciones CHECK y las llaves foráneas, y el DML va dentro de procedimientos almacenados y triggers. Así, las reglas de la cevichería se aplican en el motor y no hay que repetirlas en cada aplicación que se conecte a la base de datos.

Figura 2
Diagrama entidad-relación de Cevicheria_DB



Nota. Elaboración propia con SQL. Script: 01_BaseDatos.sql.

1.2.1. Objetos de automatización

La Tabla 6 documenta el propósito, los parámetros y el comportamiento ante excepciones de cada objeto. No incluye usp_RegistrarError, un procedimiento auxiliar que solo guarda en Log_Errores el número y el mensaje del error capturado. El código completo está en el repositorio y en el anexo de código fuente.

Tabla 6
Procedimientos, funciones y triggers implementados

Objeto	Propósito y parámetros	Comportamiento ante excepciones
sp_IngresarPedido	Registra un pedido con sus líneas. Parámetros: @id_pedido, @id_cliente (NULL → 9999), @id_empleado, @canal, @metodo_pago y una tabla @lineas (TVP).	TRY/CATCH con transacción. Cada línea usa SAVE TRANSACTION: si una línea falla, solo esa línea se revierte (ROLLBACK a savepoint) y se registra; si no queda ninguna línea válida, ROLLBACK total.
sp_ActualizarSalario	Cambia el salario de un empleado. Parámetros: @id_empleado, @nuevo_salario.	Valida existencia y que el salario no sea menor a la RMV (S/ 1 130); ante error hace ROLLBACK, registra en Log_Errores y devuelve el mensaje.

Objeto	Propósito y parámetros	Comportamiento ante excepciones
sp_CargaMasivaClientes	BULK INSERT a staging, deduplica (R1), invalida DNI $\neq$ 8 dígitos (R5) e inserta en Cliente. Parámetro: @ruta.	Transacción única; ante error, ROLLBACK, registro en Log_Errores y relanza el error (THROW).
sp_CargaMasivaPedidos	BULK INSERT de pedidos y detalle; aplica R1 a R4 y R6. Parámetros: @ruta_pedidos, @ruta_detalle.	Dos savepoints (sv_pedidos, sv_insert): si falla la inserción de pedidos se revierte solo ese tramo; rechazos individuales van a Log_Errores y el resumen a Log_Auditoria.
fn_CategoriaVenta (escalar)	Clasifica un monto en “Venta Grande” ($> S/ 150$) o “Venta Pequeña”.	Devuelve “Sin monto” si el valor es NULL.
fn_ValidarDNI (escalar)	Devuelve 1 si el DNI tiene exactamente 8 dígitos.	Devuelve 0 ante NULL o caracteres no numéricos.
fn_VentasPorCategoria (tabla)	Ventas, unidades y margen por categoría en un rango de fechas.	Rango vacío devuelve 0 filas; reutilizable por Power BI y por el rol analista.
trg_ValidarMontoPedido (integridad)	AFTER INSERT/UPDATE en Pedido: bloquea monto_total ≤ 0 .	ROLLBACK, registro del intento en Log_Auditoria y THROW 51030; seguro para operaciones multifila.
trg_ValidarDetalle (integridad)	AFTER INSERT/UPDATE en Detalle_Pedido: exige subtotal = cantidad $\times$ precio.	ROLLBACK, registro y THROW 51031.
trg_AuditoriaEmpleado (auditoría)	AFTER UPDATE/DELETE en Empleado: registra valor anterior y nuevo de salario y cargo, y las bajas.	Solo registra filas con cambio real (compara inserted y deleted).
trg_AuditoriaPedido (auditoría)	AFTER UPDATE/DELETE en Pedido: registra cambios de monto y anulaciones.	No audita el ajuste técnico posterior al alta (monto provisional 0.01).
trg_ProtegerLogAuditoria	INSTEAD OF UPDATE/DELETE en Log_Auditoria: el log es de solo inserción.	THROW 51040 ante cualquier intento de modificación o borrado.

Nota. TVP = parámetro con valores de tabla. Las reglas de carga R1 a R6 se describen en la Tabla 7. Elaboración propia.

1.2.2. Reglas de carga y resultados

Para la carga masiva se usan tablas de paso (staging) con todas las columnas en texto, de modo que ningún valor sucio detenga el BULK INSERT; recién después se

valida con TRY_CAST y TRY_CONVERT. La Tabla 7 muestra las reglas aplicadas a los CSV crudos y sus resultados.

Tabla 7

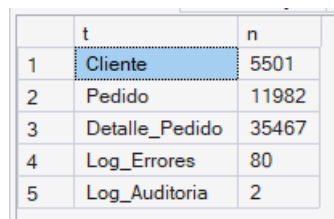
Reglas de validación de la carga masiva y registros afectados

Regla	Descripción	Tratamiento	Registros
R1	Duplicados por llave (Pedido / Cliente)	Se descartan (ROW_NUMBER = 1)	38 / 45
R2	Pedido sin id_cliente	Se asigna 9999 (S1)	1 080
R3	Fecha no convertible	Fecha de carga y fecha_imputada = 1 (S2)	78
R4	monto_total ≤ 0	Se rechaza a Log_Errores	18
R5	DNI de cliente ≠ 8 dígitos	dni = NULL; el cliente se conserva	32
R6	Detalle sin pedido padre	Se rechaza a Log_Errores	52
—	Resultado final en tablas	11 982 pedidos, 35 467 líneas y 5 501 clientes (incluye el 9999)	80 filas en Log_Errores

Nota. Los rechazos de las reglas R4 y R6 suman 70 filas de Log_Errores. Elaboración propia con datos de Log_Carga_Calidad.csv.

Figura 3

Ejecución de la carga masiva en SQL Server



	t	n
1	Cliente	5501
2	Pedido	11982
3	Detalle_Pedido	35467
4	Log_Errores	80
5	Log_Auditoria	2

Nota. Resultado devuelto por sp_CargaMasivaPedidos y conteo de tablas tras ejecutar 09_Carga_Datos.sql.

1.2.3. Pruebas y log de auditoría

Tabla 8

Casos de prueba de procedimientos, triggers y funciones (05_Pruebas_Validacion.sql)

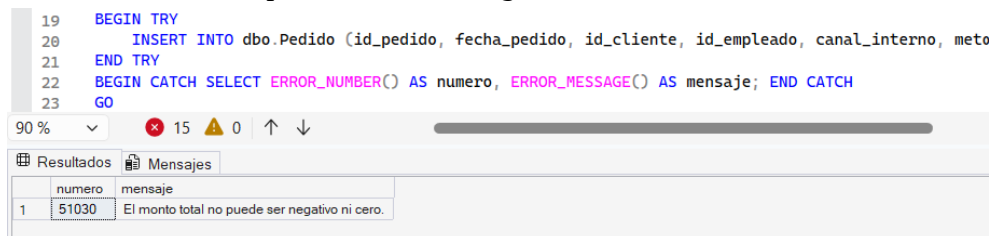
Caso	Acción	Resultado esperado
P1	sp_IngresarPedido con cliente NULL y una línea con plato inexistente	Pedido con id_cliente 9999; se guardan 2 líneas; la línea inválida se revierte por savepoint y queda en Log_Errores
P2	sp_IngresarPedido sin líneas válidas	ROLLBACK total: 0 filas del pedido; resultado “ERROR”

Caso	Acción	Resultado esperado
P3	INSERT en Pedido con monto -10	trg_ValidarMontoPedido revierte y lanza el error 51030; queda fila “Intento bloqueado” en Log_Auditoria
P4	sp_ActualizarSalario con 1 800, con 500 y con empleado inexistente	El primero actualiza y audita valor anterior y nuevo, los otros dos hacen ROLLBACK y se registran
P5	DELETE sobre Log_Auditoria	Error 51040: el log es de solo inserción
P6	fn_CategoriaVenta(200), fn_ValidarDNI('1234567'), fn_VentasPorCategoria	“Venta Grande”; 0; ventas por categoría del primer trimestre de 2026

Nota. El resultado esperado se deduce de la lógica de cada objeto, la evidencia de ejecución se adjunta en las Figuras 4 a 6. Elaboración propia.

Figura 4

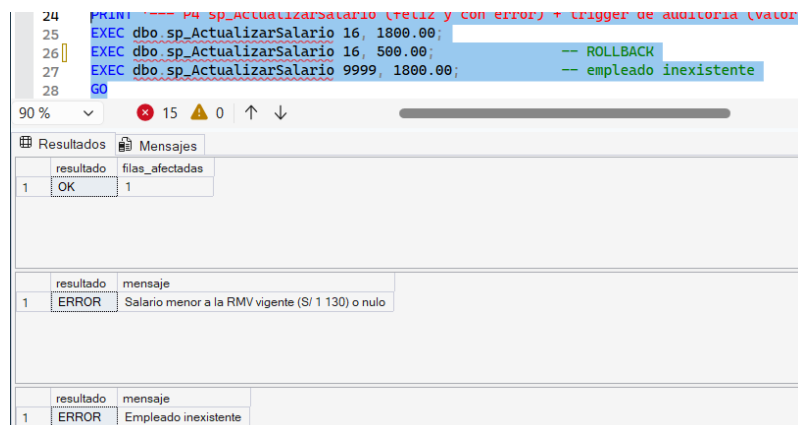
Prueba de control de excepciones: monto negativo rechazado



Nota. El trigger de integridad revierte la transacción con un monto de -10.00.

Figura 5

Prueba de actualización salarial



Nota. Confirmación del motor tras ejecutar el procedimiento de actualización de salario.

Figura 6

Registro del trigger de auditoría en Log_Auditoria

	id_log	fecha_hora	usuario	accion	tabla_afectada	valor_anterior	valor_nuevo
1	4	2026-09-30 22:21:34	LAPTOP-G5KU4MHGHP	Cambio de salario del empleado ID 16	Empleado	1502.62	1800.00
2	3	2026-09-30 22:20:16	LAPTOP-G5KU4MHGHP	Intento bloqueado: monto_total <= 0 (1 fila/s)	Pedido	NULL	900003
3	2	2026-09-30 21:53:10	LAPTOP-G5KU4MHGHP	Carga masiva pedidos	Pedido	NULL	ins_pedidos=11982; ins_detalle=35467; dup=38; cliente_pa...
4	1	2026-09-30 21:53:02	LAPTOP-G5KU4MHGHP	Carga masiva clientes: insertados=5500, duplicad...	Cliente	NULL	C:\Grupo3_CliIN1021P_EF_ENTREGA\Grupo3_CliIN1021P...

Nota. Consulta a Log_Auditoria con valor anterior y nuevo del salario.

Figura 7
Evidencia de las pruebas de Log_Errores

```
38 SELECT TOP 20 * FROM dbo.Log_Errores ORDER BY id_error DESC;
39 GO
40
```

id_error	fecha_hora	procedimiento	numero_error	mensaje	tabla	id_registro
82	2026-09-30 22:21:34	sp_ActualizarSalario	51010	Empleado inexistente	Empleado	9999
81	2026-09-30 22:21:34	sp_ActualizarSalario	51011	Salario menor a la RMV vigente (S/ 1 130) o nulo	Empleado	16
80	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	34666
79	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	34665
78	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	34664
77	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	33813
76	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	33577
75	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	33576
74	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	33575
73	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28719
72	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28718
71	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28717
70	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28716
69	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28685
68	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28684
67	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28683
66	2026-09-30 21:53:09	sp_CargaMasivaPedidos	51021	detalle huérfano: pedido padre rechazado	Detalle_Pedido	28682

Nota. Salida de Log_Errores.

El componente más incierto era si un trigger podía interceptar y revertir una transacción. Lo probamos con un monto de -10.00 y el motor lo rechazó, la ampliamos con los casos P1 a P6, que cubren transacciones, savepoints, auditoría y que el log no se pueda modificar.

1.2.4. Justificación de diseño frente a alternativas

Comparamos nuestra opción (lógica en la base de datos, con procedimientos y triggers) con tres alternativas:

- (A) Validar en una aplicación intermedia.
- (B) Validar solo en la interfaz.
- (C) Hacerlo todo en el ETL. La Tabla 9 muestra el resultado por criterio.

Tabla 9
Comparación de alternativas para el bloque de automatización

Criterio	Elegida: T-SQL en la BD	A: capa de aplicación	B: interfaz (frontend)	C: solo ETL
Seguridad e integridad	Ganadora. La regla se aplica a todo acceso, incluso desde SSMS (P3)	Se evade con acceso directo a la BD	Descartada: se evade con acceso directo a la BD	Los datos sucios ya entraron a la BD

Criterio	Elegida: T-SQL en la BD	A: capa de aplicación	B: interfaz (frontend)	C: solo ETL
Tráfico de red	1 llamada por carga (12 038 pedidos)	Hasta 12 038 viajes de ida y vuelta	Sin validación en servidor	1 archivo, pero sin control transaccional
Trazabilidad	Ganadora. Trigger audita cualquier modificación.	Solo audita lo que pasa por la aplicación	Ninguna	Solo audita el proceso ETL
Mantenibilidad y acoplamiento	Reglas centralizadas; Power BI y otras apps las reutilizan	Reglas duplicadas por aplicación	Duplicadas por pantalla	Acoplada al script
Costo de CPU del servidor	Mayor carga en la BD	Menor carga en la BD	Menor	Menor

Nota. Elaboración propia. Los tres criterios de mayor peso para el negocio fueron seguridad, trazabilidad y tráfico de red.

Condición bajo la cual una alternativa sería preferible: La alternativa A

(lógica en aplicación) sería mejor si el servidor de base de datos tuviera muy poca CPU o si hubiera muchos locales con reglas distintas, porque el procesamiento pesado saturaría la BD.

Trigger frente a procedimiento: El procedimiento controla cómo entra un dato y devuelve resultados; el trigger asegura que toda modificación deje huella. Por eso usamos procedimientos para la ingesta y triggers para integridad y auditoría, y no uno en lugar del otro.

1.3. Implementación de políticas de seguridad y cumplimiento normativo

Diseñamos la seguridad sobre tres principios: mínimo privilegio, segregación de funciones y minimización de datos personales. Los datos de clientes (DNI, correo, teléfono) y del personal (DNI, salario) son datos personales protegidos por la Ley N.º 29733 (2011), cuyo principio de seguridad (art. 9) exige medidas técnicas, organizativas

y legales apropiadas. Además, alineamos las medidas con los controles de la norma ISO/IEC 27001:2022.

1.3.1. Roles y segregación de privilegios

Tabla 10

Roles de base de datos y privilegios diferenciados

Rol	Privilegios concedidos	Restricciones (DENY)	Justificación de la segregación
Rol_Administrador	CRUD sobre tablas de negocio; EXECUTE; SELECT en Log_Auditoria; UNMASK	INSERT, UPDATE y DELETE sobre Log_Auditoria	Gestiona el negocio pero no puede alterar la evidencia de auditoría
Rol_Analista Datos	SELECT en vistas seudonimizadas, Pedido, Detalle, Plato y fn_VentasPorCategoria	SELECT en Cliente, Empleado, Log_Auditoria y Log_Errores	Hace BI sin ver DNI, correos ni salarios
Rol_Auditor	SELECT en Log_Auditoria y Log_Errores	Todo acceso a Pedido, Cliente y Empleado	Controla sin conocer datos del negocio (separación de tareas)
Rol_Vendedor	EXECUTE en sp_IngresarPedido; SELECT en Plato y en 4 columnas de Cliente	UPDATE y DELETE en Pedido; SELECT en Empleado y en el log	Registra ventas solo mediante el procedimiento validado
Rol_Cliente	SELECT en vw_MisPedidos y Plato, con filtro por fila (RLS)	Cliente, Empleado y log	Cada cliente ve únicamente su historial
Rol_Cocina	SELECT en vw_PedidosCocina (platos y cantidades del día)	Pedido y Cliente	Ve qué preparar, sin montos ni datos de clientes

Nota. Script: 06_Roles_Privilegios_RLS.sql. El enmascaramiento dinámico (DDM) oculta correo, teléfono, DNI del empleado y salario a quien no tiene UNMASK; la política de seguridad seg.PoliticaPedidoCliente aplica el filtro por fila (Microsoft, s. f.-a, s. f.-b).

Aparte de los roles, el DW nunca guarda el DNI en claro: vw_DW_Cliente y el ETL calculan un hash SHA-256 con sal y dejan fuera nombre, correo, teléfono y salario. Si hubiera un incidente en el entorno analítico, identificar a las personas sería mucho más difícil, aunque el hash tiene límites que se discuten en la sección 1.8.

Figura 8
Pruebas de privilegios con EXECUTE AS

```

87 PRINT '--- T1 analista: puede leer vista pseudonimizada; NO puede leer Empleado (salarios)';
88 EXECUTE AS USER = 'u_analista'; SELECT TOP 3 * FROM dbo.vw_Dm_Cliente;
89 BEGIN TRY SELECT TOP 1 salario_mensual FROM dbo.Empleado; END TRY BEGIN CATCH SELECT ERROR_MESSAGE() AS denegado; END CATCH REVERT;
90 PRINT '--- T2 vendedor: no puede borrar pedidos ni leer auditoría';
91 EXECUTE AS USER = 'u_vendedor'; BEGIN TRY DELETE FROM dbo.Pedido WHERE id_pedido = 1; END TRY BEGIN CATCH SELECT ERROR_MESSAGE() AS denegado; EN
92 PRINT '--- T3 cliente 1 solo ve sus pedidos (RLS)';
93 EXECUTE AS USER = 'u_cliente1'; SELECT COUNT(*) AS mis_pedidos, MIN(id_pedido) mn FROM dbo.vw_MisPedidos; REVERT;
94 PRINT '--- T4 auditor ve logs pero no ventas';
95 EXECUTE AS USER = 'u_auditor'; SELECT TOP 3 * FROM dbo.Log_Auditoria; BEGIN TRY SELECT TOP 1 * FROM dbo.Pedido; END TRY BEGIN CATCH SELECT ERROR
96 PRINT '--- T5 enmascaramiento: vendedor/admin leen Cliente con DNI ofuscado o completo según UNMASK';
97 EXECUTE AS USER = 'u_admin'; SELECT TOP 2 id_cliente, dni, email FROM dbo.Cliente; REVERT; -- email visible por UNMASK del administrador
98 GO

```

90 % No se encontraron problemas. Línea: 85, Carácter: 71 SPC LF

Resultados Mensajes

id_cliente	dni_hash	distrito	tipo_cliente	fecha_registro
1	66B5188592AD0F53E8A43CD161201A680CC027910CF350C	Cajamarca	Nuevo	2026-06-07
2	80914DD0651AED776762FA114F6DBEACAD4191E3A20727B05	Cajamarca	Frecuente	2025-12-31
3	B62853E7468E45AC73F04598D1879470811764B8FFA4DC0C	Cajamarca	Frecuente	2025-03-13

denegado

1 The SELECT permission was denied on the object 'E'.

denegado

1 The SELECT permission was denied on the object 'P'.

mis_pedidos	mn
3	8390

id_log	fecha_hora	usuario	accion	tabla_afectada	valor_anterior	valor_nuevo
1	2026-09-30 21:53:02	LAPTOP-Q5KJ4MHQ3HP	Carga masiva clientes: insertados=5500, duplicados=	Cliente	NULL	C:\Grupo3_ORN1021P_EF_ENTREGA\Grupo3_ORN1021P...
2	2026-09-30 21:53:10	LAPTOP-Q5KJ4MHQ3HP	Carga masiva pedidos	Pedido	NULL	ms_pedidos=11982, vw_detalle=35487, dup=38, cliente_pa...
3	2026-09-30 22:20:16	LAPTOP-Q5KJ4MHQ3HP	Intento bloqueado: monto_total <= 0 (1 fila/s)	Pedido	NULL	800003

denegado

1 The SELECT permission was denied on the object 'P'.

id_cliente	dni	email
1	45593336	NULL
2	29479906	jose2@correo.pa

Nota. Pruebas T1 a T5 del script de roles: el analista no puede leer salarios, el vendedor no puede borrar pedidos, el cliente 1 solo ve sus pedidos y el auditor no lee Pedido.

1.3.2. Análisis de plan de ejecución e índices

Analizamos dos consultas críticas del tablero: Q1 (ventas de una semana) y Q2 (los diez platos más vendidos de un mes), con los 11 982 pedidos y 35 467 líneas cargados. Los tiempos de la Tabla 11 se midieron en SQLite como motor de referencia, con los mismos datos; las lecturas lógicas y la CPU de SQL Server se obtuvieron con SET STATISTICS IO, TIME ON al ejecutar 07_Indices_Rendimiento.sql.

Tabla 11
Métricas antes y después del índice (11 982 pedidos y 35 467 líneas)

Consulta	Estado	Operador (motor de referencia)	Tiempo mediano (ms)	SQL Server: lecturas lógicas / CPU (ms)
Q1 ventas por semana	Antes (sin índice)	SCAN de tabla	0.587	89/0
	Después (IX_Pedido_Fecha con INCLUDE)	SEARCH con índice cubriente	0.033 (−94 %)	2/0
Q2 top 10 platos del mes	Antes (sin índices)	SCAN de Pedido; índices automáticos temporales	14.261	177/15
	Después (IX_Pedido_Fecha + IX_Detalle_Pedido)	SEARCH por fecha y por id_pedido (índice cubriente)	2.136 (−85 %)	94/16

Nota. Mediana de 200 ejecuciones por consulta en SQLite (indice_referencia_sqlite.py). Los valores de SQL Server difieren, pero la mejora cualitativa (Scan → Seek) es la misma. El índice con columnas incluidas evita el *Key Lookup*, por lo que se declaró “cubriente”.

Figura 9

Plan de ejecución real con 11 982 pedidos en SQL Server

```
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
  CPU time = 16 ms, elapsed time = 20 ms.

SQL Server Execution Times:
  CPU time = 15 ms, elapsed time = 3 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 18 ms.

(7 filas afectadas)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0.
Table 'Pedido'. Scan count 1, logical reads 89, physical reads 1, page server reads 0, read-ahead reads 87,

(1 fila afectada)

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 72 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0.
Table 'Pedido'. Scan count 1, logical reads 89, physical reads 0, page server reads 0, read-ahead reads 0, ;

SQL Server Execution Times:
  CPU time = 32 ms, elapsed time = 28 ms.
SQL Server parse and compile time:
  CPU time = 14 ms, elapsed time = 14 ms.

(7 filas afectadas)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0.
Table 'Pedido'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, ;

(1 fila afectada)

SQL Server Execution Times:
  CPU time = 16 ms, elapsed time = 84 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.

Hora de finalización: 2026-09-30T22:57:28.8949627-05:00
```

Nota. Planes de Q1 antes y después del índice IX_Pedido_Fecha.

1.3.3. Política de respaldo y restauración

Como en horas punta hay muchas transacciones, elegimos una política FULL + DIFFERENTIAL + LOG. El diferencial diario acorta la restauración frente a encadenar solo respaldos de log, y el log cada 15 minutos limita cuántos datos se pueden perder. El script incluye RESTORE VERIFYONLY con CHECKSUM, una restauración de prueba en otra base (Cevicheria_DB_TEST) comparando conteos y un trabajo de SQL Server Agent para el FULL semanal.

Tabla 12

Política de respaldo, RPO y RTO

Tipo	Frecuencia	Objetivo	Verificación
FULL	Domingo 03:00	Punto de partida de la cadena de restauración	RESTORE VERIFYONLY ... WITH CHECKSUM

Tipo	Frecuencia	Objetivo	Verificación
DIFFERENTIAL	Lunes a sábado 03:00	Reduce el RTO (solo se aplica el último diferencial)	RESTORE VERIFYONLY ... WITH CHECKSUM
LOG	Cada 15 min (10:00–23:00)	$RPO \leq 15$ minutos	Restauración de prueba con RECOVERY
Objetivos	$RPO \leq 15$ min; $RTO \leq 30$ min	Continuidad del servicio de la cevichería	Conteo de Pedido original = restaurada

Nota. Script: 08_Backup_Restore.sql. Los objetivos RPO y RTO son metas de diseño del equipo.

Figura 10

Prueba de restauración FULL + DIFFERENTIAL + LOG

```

34 ALTER DATABASE Cevicheria_DB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
35 RESTORE DATABASE Cevicheria_DB FROM DISK = 'C:\Backups\Cevicheria_DB_FULL.bak' WITH NORECOVERY, REPLACE;
36 RESTORE DATABASE Cevicheria_DB FROM DISK = 'C:\Backups\Cevicheria_DB_DIFF.bak' WITH NORECOVERY;
37 RESTORE LOG Cevicheria_DB FROM DISK = 'C:\Backups\Cevicheria_DB_LOG.trn' WITH RECOVERY;
38 ALTER DATABASE Cevicheria_DB SET MULTI_USER;

```

90 % 1 0

Mensajes

Processed 1904 pages for database 'Cevicheria_DB', file 'Cevicheria_DB' on file 1.
 Processed 2 pages for database 'Cevicheria_DB', file 'Cevicheria_DB_log' on file 1.
 RESTORE DATABASE successfully processed 1906 pages in 0.182 seconds (81.795 MB/sec).
 Processed 72 pages for database 'Cevicheria_DB', file 'Cevicheria_DB' on file 1.
 Processed 2 pages for database 'Cevicheria_DB', file 'Cevicheria_DB_log' on file 1.
 RESTORE DATABASE successfully processed 74 pages in 0.049 seconds (12.760 MB/sec).
 Processed 0 pages for database 'Cevicheria_DB', file 'Cevicheria_DB' on file 1.
 Processed 8 pages for database 'Cevicheria_DB', file 'Cevicheria_DB_log' on file 1.
 RESTORE LOG successfully processed 8 pages in 0.004 seconds (19.625 MB/sec).

Hora de finalización: 2026-10-01T14:13:50.1969710-05:00

Nota. Elaboración propia.

1.3.3.1. Almacenamiento de respaldos en la nube mediante Google Drive

Como medida adicional, los respaldos generados desde SQL Server se guardan también en Google Drive. Así tenemos una copia fuera del servidor por si falla el equipo, se pierde información o se daña el almacenamiento local.

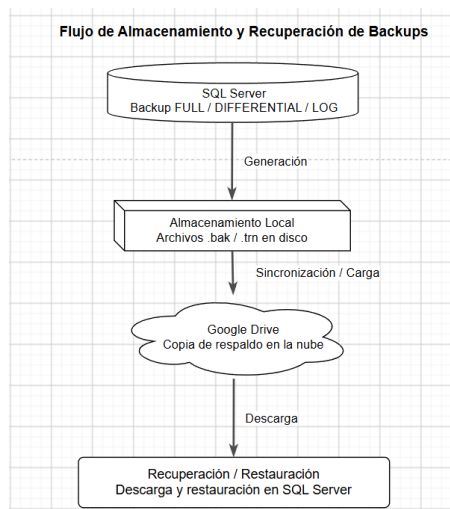
Los respaldos FULL, DIFFERENTIAL y LOG se generan según la política anterior. Primero se guardan en una carpeta local exclusiva para respaldos y luego se suben a Google Drive, con esta estructura: Google Drive → Cevichería La Esquina → Backups → Año → Mes → Tipo de respaldo. Por ejemplo:

- Backups/2026/Octubre/FULL
- Backups/2026/Octubre/DIFFERENTIAL
- Backups/2026/Octubre/LOG

El flujo completo es: SQL Server → generación del respaldo → carpeta local → subida a Google Drive → verificación del archivo → restauración cuando sea necesario (Figura 11). La copia en Google Drive es un respaldo externo que complementa al almacenamiento local. También hay que comprobar que el archivo subido se pueda usar en una prueba de restauración, para asegurar la disponibilidad ante incidentes.

Figura 11

Flujo de almacenamiento de respaldos en Google Drive



Nota. El almacenamiento en Google Drive se plantea como mecanismo de respaldo externo para complementar las copias locales y fortalecer la disponibilidad y recuperación de la información de la base de datos. Elaboración propia.

1.3.4. Lista de verificación normativa y consecuencias

Tabla 13

Verificación de cumplimiento de la Ley N.º 29733 y de ISO/IEC 27001:2022

Norma y artículo o control	Medida implementada	Evidencia
Ley 29733, art. 8 (calidad)	Datos veraces y exactos: restricciones CHECK, reglas R1 a R6 y conciliación en el ETL	Tabla 3 y Tabla 7; ETL T07 y T08
Ley 29733, art. 9 (seguridad) y art. 11 (nivel de protección adecuado)	Roles, DENY, DDM y RLS como medidas técnicas	Tabla 10; pruebas T1 a T5
Ley 29733, art. 16 (seguridad del tratamiento)	Respaldos con verificación y restauración probada; auditoría de cambios	Tabla 12; Log_Auditoria
Ley 29733, art. 17 (confidencialidad)	Seudonimización del DNI y exclusión de datos personales del DW	ETL T05; vw_DW_Cliente

Norma y artículo o control	Medida implementada	Evidencia
ISO 27001:2022 A.5.15 y A.5.18 (control y derechos de acceso)	Accesos por rol y revisión periódica de membresías	Script de roles
ISO 27001:2022 A.5.3 (segregación de funciones)	El auditor no lee datos del negocio; el administrador no altera el log	Rol_Auditor; trg_ProtegerLogAuditoria
ISO 27001:2022 A.8.11 y A.8.24 (enmascaramiento y criptografía)	DDM y hash SHA-256 con sal	DDM; ETL T05
ISO 27001:2022 A.8.13 y A.8.15 (copias de seguridad y registro)	FULL + DIFF + LOG y Log_Auditoria	Tabla 12; Tabla 6

Nota. El reglamento vigente es el D.S. N.º 016-2024-JUS. Por ello la verificación se basa en los artículos de la Ley (2011) y en controles de ISO/IEC (2022).

Consecuencias sobre los afectados:

(1) Sin segregación de roles ni enmascaramiento, cualquier usuario con permiso de lectura vería 5 468 DNI válidos y 5 335 correos de clientes, con riesgo de suplantación de identidad y phishing dirigido.

(2) Los salarios de los 21 trabajadores con remuneración declarada quedarían a la vista de cualquier analista o vendedor, lo que podría generar conflictos laborales e incluso extorsión.

(3) Sin un log de solo inserción, un fraude interno, como bajar un pedido de S/ 90 a S/ 9, no dejaría rastro y la sanción podría caer injustamente sobre el trabajador de turno. Estas consecuencias se retoman en la sección 1.8.

1.4. Integración SQL–NoSQL

SQL Server maneja bien las transacciones críticas y estructuradas (ventas, pagos, salarios), donde se necesita consistencia ACID. MongoDB, en cambio, es una base orientada a documentos con esquema flexible y escalabilidad horizontal, útil para datos semiestructurados (Sarasa Cabezuelo, 2016). Implementamos dos colecciones:

CatalogoMenu, con la carta y atributos variables (alérgenos, temporada, opciones

vegetarianas), y **Resenas**, con 5 200 reseñas importadas desde JSON. En las reseñas, el 55 % incluye id_cliente, el 60 % etiquetas, el 25 % respuesta del local, el 30 % votos de utilidad y el 15 % un subdocumento de visita; es decir, cada documento tiene una forma distinta, y en SQL eso obligaría a usar columnas nulas o tablas adicionales.

1.4.1. Operaciones CRUD documentadas

El script mongodb_resenas.js contiene las operaciones CREATE (insertMany), READ (find con \$in, \$all y \$exists), UPDATE (\$set, \$push, updateMany) y DELETE (deleteOne, deleteMany), además de agregaciones (\$group, \$match, \$sort) e índices compuestos sobre plato y puntuacion. Las Figuras 12 a 14 muestran la ejecución de las operaciones básicas sobre CatalogoMenu.

Figura 12
CRUD en MongoDB: creación

```
> db.CatalogoMenu.insertMany([
  { plato_principal: "Ceviche Clásico", precio_base: 28.00, ingredientes_extra: ["Chifles", "Camote extra"], alergenos: ["pescado"],
    resenas_clientes: [{ usuario: "juanito99", puntuacion: 5, comentario: "El pescado súper fresco." } ] },
  { plato_principal: "Leche de Tigre Especial", precio_base: 24.00, temporada: "verano", alergenos: ["pescado", "mariscos" ] },
  { plato_principal: "Ceviche Vegetariano", precio_base: 24.00, vegetariano: true, ingredientes_extra: ["Palta" ] })
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abeb2ec482d0fb3533dba2e'),
    '1': ObjectId('6abeb2ec482d0fb3533dba2f'),
    '2': ObjectId('6abeb2ec482d0fb3533dba30')
  }
}
```

Nota. Inserción de un documento con subdocumentos y arreglos desde mongosh.

Figura 13
CRUD en MongoDB: lectura

```
> db.CatalogoMenu.find({ ingredientes_extra: "Chifles" })
db.CatalogoMenu.find({ alergenos: { $in: ["mariscos" ] } }, { plato_principal: 1, precio_base: 1, _id: 0 })
< {
  plato_principal: 'Leche de Tigre Especial',
  precio_base: 24
}
Cevicheria_NoSQL >
```

Nota. Consulta por un elemento de un arreglo (ingredientes_extra).

Figura 14

CRUD en MongoDB: actualización y eliminación

```
> db.CatalogoMenu.updateOne({ plato_principal: "Ceviche Clásico" }, { $set: { precio_base: 26.00 }, $push: { ingredientes_extra: "Cancha" } })
db.CatalogoMenu.deleteOne({ plato_principal: "Ceviche Vegetariano" })
{
  acknowledged: true,
  deletedCount: 1
}
Cevicheria_NoSQL >
```

Nota. Actualización con \$set y eliminación del documento; el motor confirma con acknowledged: true.

1.4.2. Análisis comparativo y decisión arquitectónica

Comparamos ambos motores con cuatro criterios técnicos ponderados y los evaluamos para dos cargas distintas del negocio, porque la mejor tecnología depende del tipo de dato. Cada criterio se calificó de 1 a 5.

Tabla 14

Matriz comparativa SQL Server vs MongoDB (puntaje SQL / MongoDB)

Criterio	Peso	Ventas y pagos	Reseñas y catálogo	Resultado por criterio
1. Modelo de datos	25 %	5 / 3	2 / 5	Tablas para ventas; documentos para reseñas con campos variables
2. Consistencia	30 %	5 / 3	3 / 4	ACID es obligatorio en pagos; la consistencia eventual basta en reseñas
3. Escalabilidad	20 %	3 / 4	2 / 5	Sharding nativo favorece a MongoDB con volúmenes crecientes de reseñas
4. Latencia de consulta	25 %	4 / 3	3 / 5	Agregaciones con JOIN rinden mejor en SQL; lectura de un documento por plato, en MongoDB
Puntaje ponderado	100 %	4.35 / 3.20	2.55 / 4.70	SQL Server gana en ventas; MongoDB gana en reseñas y catálogo

Nota. Puntajes de 1 (peor) a 5 (mejor), asignados por el equipo con base en las características de cada motor; el puntaje ponderado es la suma de peso x puntaje. Elaboración propia.

Decisión arquitectónica. Elegimos una arquitectura híbrida: SQL Server para ventas, pagos, personal y auditoría, y MongoDB para reseñas y catálogo dinámico.

Descartamos SQL puro porque obligaría a usar columnas nulas o tablas auxiliares para las reseñas, y NoSQL puro porque salarios, cobros y auditoría necesitan ACID y

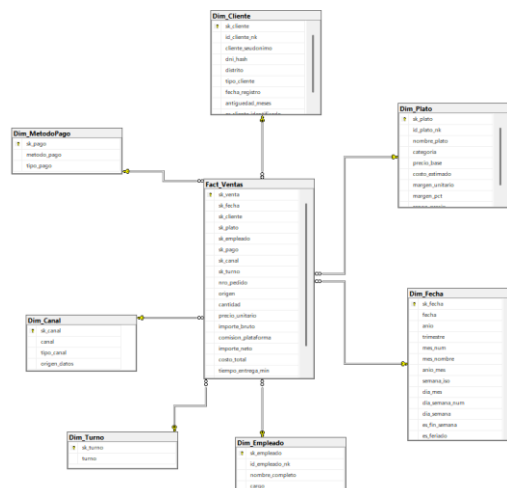
seguridad por roles. Los dos motores no se sincronizan en línea, algo que sería caro para un negocio de este tamaño, la integración se hace en el análisis donde Spark lee el mismo JSON de reseñas y lo cruza con el DW (sección 1.7).

1.5. Data Warehouse y procesos ETL

Modelamos el Data Warehouse con la metodología bottom-up de Kimball y Ross (2013): elegimos un proceso de negocio (las ventas), declaramos el grano y después definimos dimensiones y medidas. El grano es una línea de pedido, así que se puede analizar por plato, margen y canal sin perder detalle. El esquema en estrella (Figura 15) tiene una tabla de hechos (Fact_Ventas) y siete dimensiones; el DDL para SQL Server está en 12_DW_DDL.sql e incluye llaves sustitutas, claves foráneas y un índice columnstore para acelerar las agregaciones.

Figura 15

Modelo dimensional en estrella de Cevichería (Kimball)



Nota. Tabla de hechos al centro y dimensiones conformadas alrededor. nro_pedido y origen son dimensiones degeneradas. Dim_Cliente incluye los miembros especiales “Cliente de paso” y “Cliente App Delivery”. Elaboración propia con SQL.

Tabla 15

Tablas del Data Warehouse y volumen cargado

Tabla	Tipo y contenido principal	Filas
Fact_Ventas	Hecho: cantidad, importe bruto y neto, comisión, costo, margen, tiempo de entrega y calificación	40 867

Tabla	Tipo y contenido principal	Filas
Dim_Fecha	Año, trimestre, mes, semana, día, fin de semana y feriado (2025-10-01 a 2026-12-31)	457
Dim_Cliente	Seudónimo, hash de DNI, distrito, tipo, antigüedad; 2 miembros especiales	5 502
Dim_Plato	Categoría, precio, costo, margen unitario y rango de precio	40
Dim_Empleado	Nombre y cargo (sin DNI ni salario); miembro “Sin vendedor”	26
Dim_MetodoPago / Dim_Canal / Dim_Turno	Pago (5), canal con su origen de datos (5) y turno (4)	14

Nota. Volúmenes tomados de logs/etl_log.txt. Elaboración propia.

1.5.1. Proceso ETL con Python y pandas

El ETL (etl_cevicheria.py) extrae los CSV exportados desde SQL Server y el CSV externo de las plataformas de delivery, transforma los datos y carga el modelo en estrella. Cada paso deja una línea en el log de ejecución (etl_log.txt y etl_log.csv) con las filas de entrada, salida y afectadas, de modo que cada transformación se puede verificar (Tabla 16). El proceso completo demoró 1.52 segundos con 40 867 líneas de hechos (McKinney, 2010).

Tabla 16

Pasos del ETL y transformaciones aplicadas (extracto del log de ejecución)

Paso	Transformación	Filas afectadas
E01–E03	Extracción: 5 tablas exportadas de SQL Server, CSV externo de apps y feriados	53 015 + 5 460 + 20
T01	Limpieza: eliminar duplicados por order_ref en el CSV externo	60
T02	Normalización: unificar nombres de plato (tildes, mayúsculas, “cebiche”) contra el catálogo SQL	2 755
T03	Limpieza: imputar precio_bruto nulo = precio_base x cantidad x 1.10	12
T04	Tipificación de fechas ISO en pedidos y CSV externo	17 382 revisadas; 0 cambios
T05	Seudonimización de DNI (SHA-256 con sal) y exclusión de datos personales	5 468
T06	Enriquecimiento: derivar turno (almuerzo, tarde, noche) desde la hora del pedido	11 982

Paso	Transformación	Filas afectadas
T07–T08	Validación: integridad referencial y conciliación SUM(detalle) = monto_total	0 discrepancias
L01–L05	Carga de dimensiones: fecha (con 20 feriados), cliente, plato, empleado, pago, canal y turno	457 + 5 502 + 40 + 26 + 14
L06–L07	Hecho: integrar SQL Server (35 467 líneas) y CSV externo (5 400); verificar 0 llaves nulas	40 867
L08	Carga a CSV y a SQLite de validación	8 tablas

Nota. Elaboración propia.

1.5.2. Justificación metodológica: Kimball frente a Inmon

Inmon (2005) plantea un enfoque top-down: primero un almacén corporativo normalizado y luego los data marts. Kimball y Ross (2013), en cambio, construyen de abajo hacia arriba data marts dimensionales unidos por dimensiones conformadas. La Tabla 17 compara ambos enfoques para el caso de la cevichería.

Tabla 17

Comparación de Kimball (elegido) e Inmon para el caso de la cevichería

Criterio diferenciador	Kimball (bottom-up)	Inmon (top-down)	Aplicación al caso
Tiempo hasta el primer resultado	Un data mart de ventas operativo en semanas	Requiere modelar primero todo el almacén corporativo	Ciclo académico de 8 semanas y un solo proceso de negocio (ventas)
Complejidad y tamaño del equipo	Modelo en estrella, simple de consultar en Power BI	Modelo 3FN central más data marts derivados	El volumen (40 867 hechos) no exige capa 3FN adicional
Integración de fuentes	Dimensiones conformadas (plato, fecha, canal) unen SQL Server y CSV externo	Integración en el almacén central	La dimensión Dim_Canal y el miembro “Sin vendedor” integran las apps sin rediseñar
Costo y mantenimiento	Menor costo, se agregan nuevos data marts (compras, asistencia)	Mayor inversión inicial y de gobierno	Presupuesto nulo: herramientas gratuitas

Nota. Elaboración propia con base en Kimball y Ross (2013) e Inmon (2005).

Condición en que Inmon sería preferible. Si la cevichería creciera hasta ser una cadena con sistemas distintos (compras, planillas, inventario) y necesitara

consistencia corporativa, un almacén normalizado central evitaría duplicar reglas antes de construir los data marts.

1.6. Dashboard BI, KPIs y análisis OLAP

El dashboard está hecho en Power BI Desktop sobre el modelo semántico del DW: relaciones uno a muchos desde cada dimensión hacia Fact_Ventas y una tabla de fechas marcada como tal. Las medidas DAX, las jerarquías (año → trimestre → mes → día; categoría → plato) y las páginas del tablero están documentadas en 05_BI_PowerBI/medidas_DAX.md. Los valores de la Tabla 18 se calcularon sobre el DW con `olap_kpis.py` y nos sirvieron para comprobar que las medidas del tablero daban lo mismo.

1.6.1. Preguntas de negocio y KPIs

El tablero responde tres preguntas de la gerencia:

(P1) ¿qué platos y categorías generan más ventas y margen?

(P2) ¿en qué días y turnos se concentra la demanda?

(P3) ¿cuánto cuesta vender por las aplicaciones de delivery y conviene ese canal?

Tabla 18

KPIs del tablero de ventas (octubre de 2025 a septiembre de 2026)

KPI	Valor	Lectura para el negocio
Ventas netas	S/ 1 695 893.98	Ingreso después de comisiones de plataformas
Pedidos / ticket promedio	17 382 / S/ 97.57	Cada pedido deja en promedio casi S/ 98
Margen bruto	S/ 1 098 205.11 (64.8 %)	Estimado con costo de plato (supuesto S5)
Clientes recurrentes	62.3 % de 4 385	Clientes identificados con dos o más pedidos
Participación de apps	12.2 %	Ventas netas de PedidosYa y Rappi
Comisión pagada a apps	S/ 57 129.92	Costo del canal delivery externo
Calificación y tiempo (apps)	3.97 / 5; 34 min	Calidad percibida del delivery

Nota. Fuente: Fact_Ventas. Las tarjetas del tablero muestran los cinco primeros indicadores. Elaboración propia con `olap_kpis.py`.

Estos indicadores muestran cómo le va a la cevichería y sirven de base para las decisiones de la gerencia. Flores Soria (2017) explica que el análisis financiero y de costos es una herramienta para evaluar la gestión de una empresa, y eso es justo lo que permiten las ventas netas, el ticket promedio, el margen bruto y las comisiones del delivery: ver qué tan bien rinde “La Esquina” y dónde puede mejorar.

1.6.2. Gobernanza de datos de los KPIs

Tabla 19

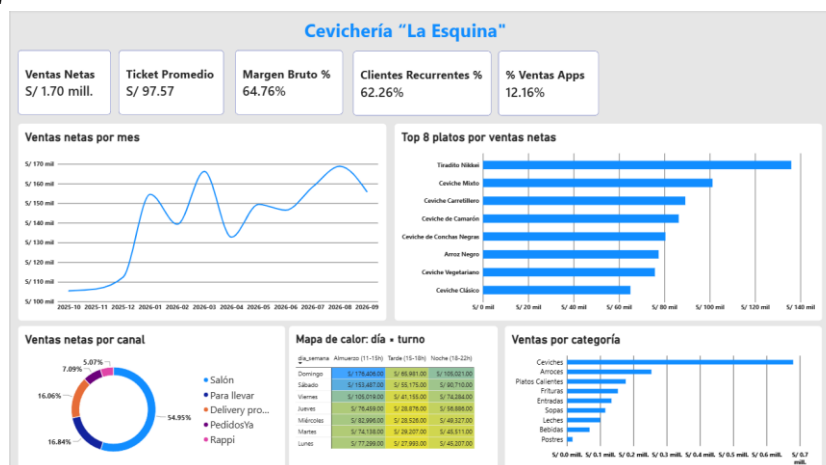
Metadatos y gobernanza de los KPIs principales

KPI	Fuente	Fórmula	Responsable	Criterio de calidad
Ventas netas	Fact_Ventas	$\Sigma \text{ importe_neto}$	Analista de datos	Conciliación con Σ monto_total: 0 discrepancias
Ticket promedio	Fact_Ventas	$\text{Ventas netas} \div \text{pedidos distintos}$	Analista de datos	nro_pedido único
Margen bruto %	Fact_Ventas y Dim_Plato	$\Sigma (\text{neto} - \text{costo}) \div \Sigma \text{ neto}$	Administrador	$\text{costo} \geq 0$; sin nulos; es una estimación
Clientes recurrentes %	Fact_Ventas y Dim_Cliente	$\text{Clientes con } \geq 2 \text{ pedidos} \div \text{clientes identificados}$	Administrador	Excluye “de paso” y “app”
% Ventas apps	Fact_Ventas	$\text{Neto (origen CSV externo)} \div \text{neto total}$	Administrador	Sin duplicados por order_ref
Calificación apps	CSV externo	Promedio de calificacion	Analista de datos	739 de 5 460 sin calificar; se ignoran

Nota. Completitud: 0 llaves nulas en el hecho; consistencia: 0 discrepancias entre detalle y cabecera; trazabilidad: `etl_log.txt` y `Log_Carga_Calidad.csv`. Elaboración propia.

Figura 16

Vista de referencia del tablero de ventas



Nota. Dashboard generado con los datos del DW (olap_kpis.py) para validar valores.

1.6.3. Operaciones OLAP

Sobre el DW ejecutamos cinco operaciones OLAP, cada una con su consulta equivalente en T-SQL (13_OLAP_Consultas.sql: ROLLUP, CUBE y GROUPING SETS) y su versión en el tablero (jerarquías y segmentadores). El roll-up resume los días en meses y trimestres (Tabla 20); el drill-down baja de la categoría Ceviches a sus platos (Tabla 21); el slice fija el canal PedidosYa; el dice restringe a la vez trimestre, categoría y método de pago (Tabla 22); y el pivot cruza día y turno.

Tabla 20

Roll-up: ventas netas y margen por trimestre (S/)

Trimestre	Ventas netas	Margen bruto	Margen %
2025-T4	324 230.00	212 362.80	65.5 %
2026-T1	460 019.28	296 913.71	64.5 %
2026-T2	428 616.66	276 654.22	64.5 %
2026-T3	483 028.04	312 274.38	64.6 %

Nota. El tercer trimestre de 2026 incluye datos hasta el 27 de septiembre. Elaboración propia.

Tabla 21

Drill-down: cinco platos con más ventas dentro de la categoría Ceviches

Plato	Ventas netas (S/)	Unidades	Margen (S/)
Tiradito Nikkei	135 945.34	3 861	89 960.83
Ceviche Mixto	101 225.25	3 050	60 782.25
Ceviche Carretillero	89 220.84	3 046	63 634.44

Plato	Ventas netas (S/)	Unidades	Margen (S/)
Ceviche de Camarón	86 321.25	2 327	54 883.48
Ceviche de Conchas Negras	80 477.33	1 958	51 538.09

Nota. La categoría Ceviches suma S/ 679 792 (40.1 % de las ventas). Elaboración propia.

Tabla 22

Dice: ventas netas del primer trimestre de 2026 por categoría y método de pago (S/)

Categoría	Plin	Yape	Total
Arroces	5 735.00	17 366.00	23 101.00
Ceviches	17 678.00	45 374.00	63 052.00
Total	23 413.00	62 740.00	86 153.00

Nota. Filtros: año 2026, trimestre 1, categorías Ceviches y Arroces, métodos Yape y Plin. Elaboración propia.

Figura 17

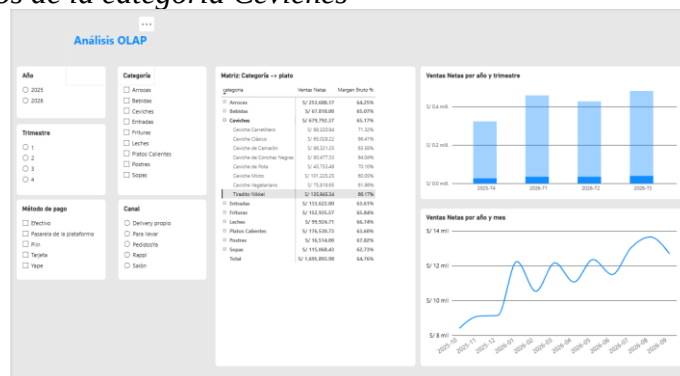
Roll-up: ventas netas por categoría y por trimestre



Nota. Captura de Power BI Desktop, página "OLAP" del tablero. Matriz con las nueve categorías colapsadas y gráfico de columnas por trimestre, sin filtros activos. El total general es S/ 1 695 893.98, con un margen bruto de 64.76 %. Elaboración propia.

Figura 18

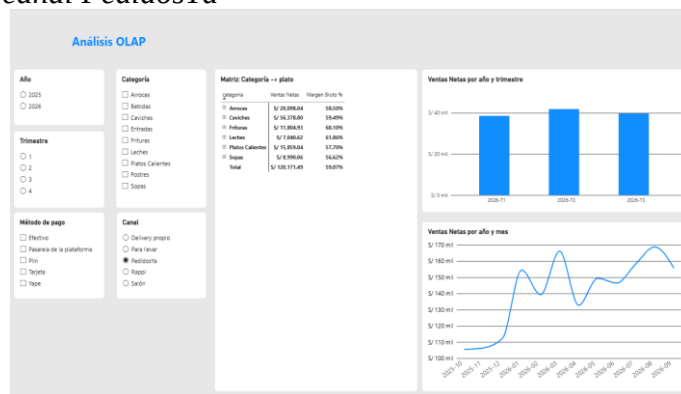
Drill-down: platos de la categoría Ceviches



Nota. Captura de Power BI Desktop, página "OLAP" del tablero. La categoría Ceviches se expande a sus platos mediante las flechas del visual; suma S/ 679 792.37 y el plato con más ventas es el Tiradito Nikkei (S/ 135 945.34). Elaboración propia.

Figura 19

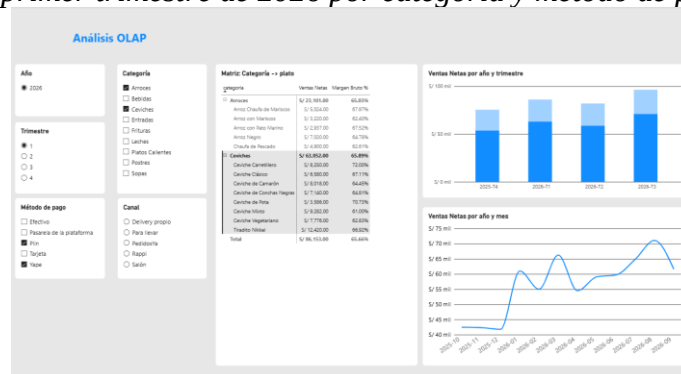
Slice: ventas del canal PedidosYa



Nota. Captura de Power BI Desktop, página "OLAP" del tablero. Se fija una sola dimensión (canal = PedidosYa) y la matriz y los gráficos de fechas se recalculan solo para ese canal. Elaboración propia.

Figura 20

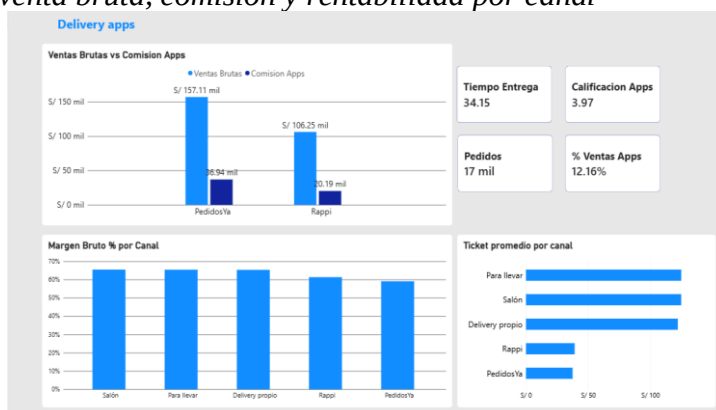
Dice: ventas del primer trimestre de 2026 por categoría y método de pago



Nota. Captura de Power BI Desktop, página "OLAP" del tablero. Se restringen varias dimensiones a la vez: año 2026, trimestre 1, categorías Arroces y Ceviches, y métodos de pago Plin y Yape. El total de S/ 86 153.00 coincide con la Tabla 22. Elaboración propia.

Figura 21

Delivery apps: venta bruta, comisión y rentabilidad por canal



Nota. Captura de Power BI Desktop, página "Delivery apps" del tablero. Compara la venta bruta con la comisión retenida por PedidosYa (S/ 36 938.21) y Rappi (S/ 20 191.71), el margen bruto porcentual de cada canal y el ticket promedio por pedido. Elaboración propia.

Respuestas a las preguntas de negocio:

(P1) Los ceviches concentran el 40.1 % de las ventas y el Tiradito Nikkei es el plato líder (S/ 135 945). Los márgenes por categoría son parecidos (entre 62.7 % y 67.8 %), así que lo que define qué categoría rinde más es el volumen y no el margen unitario.

(P2) El 58 % de las ventas de los canales propios ocurre de viernes a domingo; el domingo al almuerzo llega a S/ 176 406, frente a S/ 153 487 del sábado, y de lunes a jueves la noche y la tarde son los turnos más débiles.

(P3, Figura 21) Las aplicaciones aportan el 12.2 % de las ventas netas, pero retienen el 21.7 % de la venta bruta como comisión (S/ 57 129.92), y su margen (60 %) queda 5.4 puntos por debajo del de los canales propios (65.4 %). El delivery propio y el salón son más rentables por pedido, aunque las apps amplían el alcance.

1.7. Análisis Big Data con Apache Spark y evaluación de escalabilidad

Configuramos Apache Spark 4.2.0 con JDK 21 siguiendo los pasos del cuaderno de clase: instalar el JDK, descargar Spark, definir las variables JAVA_HOME y SPARK_HOME, usar findspark y crear la SparkSession. El cuaderno Cevicheria_Spark.ipynb detecta si corre en Google Colab (instala JDK y Spark) o en un equipo local (usa el paquete pyspark), y se ejecutó de principio a fin sin errores (Zaharia et al., 2016; Gómez y Macías, 2015).

Según Holmes (2018), el Big Data se refiere a grandes volúmenes de información que se almacenan y analizan con técnicas específicas para obtener datos útiles en la toma de decisiones. En este proyecto usamos Apache Spark con datos estructurados y semiestructurados del Data Warehouse, de archivos CSV y de documentos JSON, y además medimos cómo responde cuando el volumen crece.

Tabla 23

Fuentes de datos y objetos de Spark utilizados

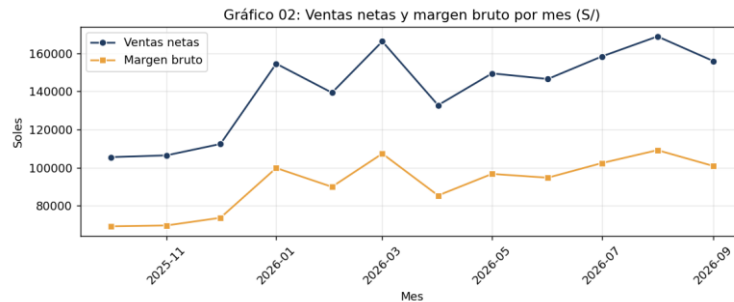
Fuente	Tipo	Objeto Spark y análisis
fact_ventas y dimensiones (DW)	CSV estructurado (40 867 filas)	DataFrame con esquema definido (StructType); describe y summary; agregaciones sum, avg, max y min por trimestre; función de ventana (top 3 platos por mes)
Mismas tablas del DW	Vistas temporales	SparkSQL: ventas por categoría con margen y ROLLUP por año, trimestre y canal
fact_ventas	Estructurado	RDD: unidades vendidas por plato con map y reduceByKey
Ventas_Delivery_Apps_Externo.csv	CSV externo (5 460 filas, sin limpiar)	DataFrame: pedidos, comisión, tiempo de entrega y calificación por plataforma y distrito
Resenas_Clientes.json	Semiestructurado (5 200 docs)	DataFrame con esquema inferido (campos opcionales y anidados), explota de etiquetas y cruce con ventas por plato
Asistencia_Personal.csv	CSV estructurado (6 000 filas)	DataFrame: horas y tardanza promedio por cargo

Nota. Todos los gráficos del cuaderno se generan con pandas, matplotlib y seaborn a partir de toPandas(), sin Power BI. Elaboración propia.

Resultados principales: Con RDD, los platos con más unidades vendidas coinciden con la categoría líder del DW. En el CSV externo, PedidosYa registra 3 236 pedidos con una comisión promedio de 23.5 % y Rappi 2 224 pedidos con 19.0 %; la calificación promedio es parecida (3.96 y 3.97) y el tiempo de entrega ronda los 34 minutos. Al cruzar las reseñas (JSON) con las ventas por plato, la correlación de Pearson entre puntuación media y ventas es $r = 0.66$: los platos mejor valorados tienden a vender más, aunque eso no prueba que una cosa cause la otra (Figura 24). Por día de la semana, el 58.2 % de las ventas netas de los canales propios se concentra de viernes a domingo y el domingo (S/ 347 408) vende más del doble que el promedio de lunes a jueves (Figura 23).

Figura 22

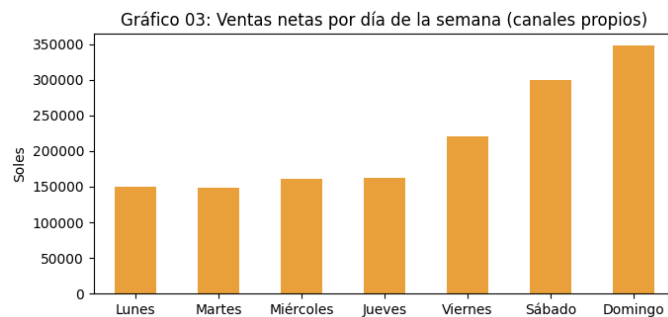
Ventas netas y margen bruto por mes calculados con Spark



Nota. Gráfico generado con pandas a partir de un DataFrame de Spark (toPandas()).

Figura 23

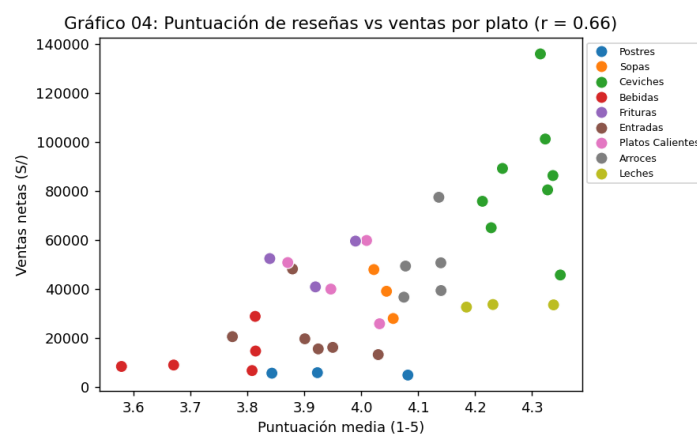
Ventas netas por día de la semana en los canales propios



Nota. Salón, para llevar y delivery propio. De viernes a domingo se concentra el 58.2 % de las ventas netas; el domingo (S/ 347 408) vende 2.2 veces más que el promedio de lunes a jueves. Gráfico generado con pandas a partir de un DataFrame de Spark. Elaboración propia.

Figura 24

Puntuación media de reseñas frente a ventas por plato



Nota. Cada punto es un plato; el color indica la categoría. Cruce de una fuente semiestructurada (JSON) con una estructurada (DW).

1.7.1. Eficiencia y escalabilidad

Para comparar la eficiencia replicamos la tabla de hechos $\times 1$, $\times 10$ y $\times 50$ y medimos la misma agregación (ventas por plato y canal, con join a la dimensión) con pandas, SQLite, SQL Server y Spark, tomando la mediana de tres ejecuciones (Tabla 24 y Figura 25). En SQL Server usamos el script 11_Benchmark_SQLServer.sql, que replica el hecho en tres tablas de prueba.

Tabla 24

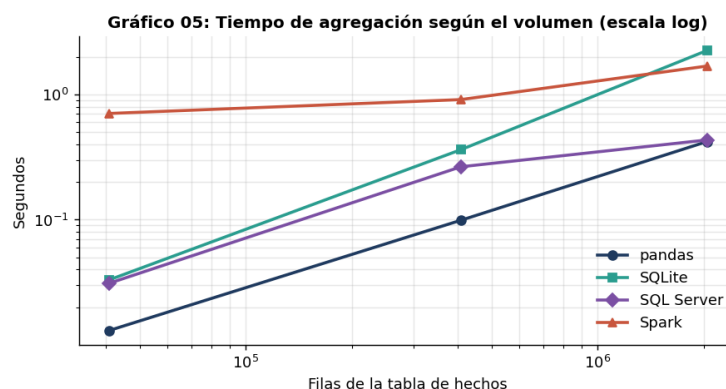
Tiempo de ejecución de la agregación según el volumen (segundos)

Filas de hechos	pandas	SQLite	SQL Server	Spark local	Spark ÷ SQL Server
40 867	0.013	0.033	0.031	0.707	22.81×
408 670	0.099	0.363	0.265	0.913	3.45×
2 043 350	0.421	2.251	0.434	1.691	3.90×

Nota. Mediana de tres ejecuciones tras una de calentamiento. pandas, SQLite y Spark (4.2.0, modo local, datos en memoria con cache) se midieron en un equipo de 1 núcleo y 4 GB de RAM; SQL Server se midió en el equipo del grupo con las tablas de prueba Bench_x1, Bench_x10 y Bench_x50 (índice sobre sk_plato) y el script 11_Benchmark_SQLServer.sql. Al ser equipos distintos, la comparación con SQL Server es orientativa. Desde el CSV original (40 867 filas), la consulta completa tardó 0.76 s en Spark y 0.14 s en pandas. Elaboración propia.

Figura 25

Tiempo de agregación según el volumen (escala logarítmica)



Nota. Con el volumen real (40 867 filas) Spark es el motor más lento; con 2 043 350 filas supera a SQLite, pero SQL Server sigue siendo más rápido en el rango probado. SQL Server se midió en otro equipo (nota de la Tabla 24).

Análisis: Con el volumen real del negocio (40 867 filas), SQL Server tarda

0.031 s y Spark 0.707 s, unas 23 veces más. La diferencia se explica por el costo fijo de

arrancar la JVM, planificar la consulta y hacer el shuffle. Al crecer el volumen la brecha baja a 3.45 veces con 408 670 filas (0.265 s frente a 0.913 s) y a 3.90 veces con 2 043 350 filas (0.434 s frente a 1.691 s): Spark mejora su posición relativa, pero en el rango probado no alcanza a SQL Server, cuyas tablas tienen índice y caben en memoria. Frente a SQLite sí gana con 2 043 350 filas (1.69 s frente a 2.25 s), y pandas es el más rápido porque el conjunto cabe en la memoria de un solo equipo y, con un solo núcleo, Spark no tiene dónde paralelizar. Como SQL Server se midió en otro equipo que pandas, SQLite y Spark, estas relaciones son orientativas. En resumen, la solución adecuada para el volumen actual ($\approx$ 12 mil pedidos al año) es SQL Server con Power BI. Spark se justifica por su lectura nativa de JSON y CSV heterogéneos y como capa de escalabilidad si el negocio creciera a una cadena de locales o sumara texto de reseñas y datos de sensores.

Tabla 25

Condiciones de uso recomendadas de cada tecnología

Condición	Tecnología recomendada	Motivo
Hasta millones de filas, consultas repetitivas, seguridad por roles y ACID	SQL Server (índices y columnstore)	Baja latencia, auditoría, RLS y DDM
Exploración individual de datos que caben en RAM	pandas	Simplicidad y menor tiempo
Cientos de millones de filas, varios nodos y formatos heterogéneos (JSON, CSV, Parquet)	Apache Spark	Paralelismo horizontal y evaluación perezosa

Nota. Elaboración propia a partir de los resultados de la Tabla 24.

Validación de viabilidad del componente más incierto. Lo más incierto era si Spark funcionaría en local con nuestros propios datos. Lo aislamos en el cuaderno y corrió sin problemas (21 celdas de código, 0 errores). El veredicto es **viable con ajuste**: Spark procesa las cuatro fuentes, pero su ventaja solo aparece con volúmenes que el negocio todavía no tiene. Por eso ajustamos la ruta: Spark queda como capa

complementaria para escalabilidad y análisis semiestructurado, y SQL Server con Power BI sigue como solución operativa.

1.8. Reflexión ética y responsabilidad profesional

Según el Código de Ética del Colegio de Ingenieros del Perú (s. f.), el ingeniero responde por las consecuencias de sus diseños sobre las personas y la sociedad, y debe resguardar la confidencialidad de la información que maneja. Aunque nuestros datos son simulados, diseñamos como si fueran reales: en un negocio de verdad, los datos de clientes y trabajadores están protegidos por la Ley N.º 29733 (2011), y los mismos patrones de diseño sirven en cualquier organización que trate datos personales. Durante el proyecto surgieron tres dilemas concretos, y la Tabla 26 resume lo que decidimos en cada uno.

Tabla 26

Dilemas éticos, decisión adoptada y consecuencias sobre los afectados

Dilema	Decisión y criterio	Consecuencia concreta sobre los afectados
D1. Utilidad analítica frente a privacidad del cliente. Identificar clientes recurrentes exige vincular pedidos con una persona, pero guardar DNI, correo y teléfono en el DW aumentaría el daño si se filtra.	Se seudonimizó el DNI (SHA-256 con sal) y se excluyeron nombre, correo y teléfono del DW. Criterio: proporcionalidad y confidencialidad (Ley 29733, arts. 8 y 17; Código de Ética del CIP).	Protege a 5 468 clientes con DNI válido de suplantación o phishing si el entorno analítico se compromete. Costo: el negocio no puede contactar a un cliente desde el DW; para una promoción debe usar la base transaccional con el rol autorizado.
D2. Sesgo en los indicadores y en los datos imputados. El cliente de paso (1 080 pedidos) y las fechas imputadas (78) distorsionan la lealtad y la tendencia.	Se declararon los supuestos S1 y S2, se marcó fecha_imputada y se excluyó al cliente de paso del KPI de recurrencia, mostrando el valor con su base (5 468 clientes identificados). Criterio: veracidad de la información y honestidad técnica.	Si se premiara solo a clientes identificados, se discriminaría a quienes pagan en efectivo sin dar sus datos, que son el 9 % de los pedidos. Evitar ocultar esa limitación previene decisiones comerciales injustas y una lectura inflada de la fidelidad (62.3 %).

Dilema	Decisión y criterio	Consecuencia concreta sobre los afectados
D3. Medición del personal. Cruzar ventas por vendedor con asistencia y tardanzas podría usarse para sancionar o vigilar a los trabajadores.	Dim_Empleado solo contiene nombre y cargo; salario y DNI no viajan al DW; el rol de analista no ve la tabla Empleado y el log de auditoría es inalterable. Criterio: justicia laboral y necesidad del dato.	Protege a 25 trabajadores de una evaluación automatizada sin contexto (por ejemplo, un domingo de poca venta no refleja su desempeño) y de exposición de su salario. Costo: la gerencia no dispone de un ranking individual listo para usar.

Nota. Elaboración propia. CIP = Colegio de Ingenieros del Perú.

Qué protege y qué podría exponer el sistema. El diseño protege los datos personales con roles, enmascaramiento, filtrado por fila y seudonimización, y protege a los trabajadores con un log que no se puede alterar. Aun así quedan riesgos. El DNI tiene solo 100 millones de combinaciones posibles, así que un hash con sal conocida podría revertirse por fuerza bruta; en este proyecto la sal está en el código solo con fines académicos, y en producción debería guardarse en una bóveda de secretos y usarse un HMAC con clave.

Qué se revisaría si el sistema se escala a nivel nacional: Si “La Esquina” creciera hasta ser una cadena con locales en distintas regiones, habría que revisar seis cosas. (a) Cambiar el hash del DNI por tokenización con clave gestionada y separar la tabla de equivalencias. (b) Inscribir el banco de datos de clientes ante la Autoridad Nacional de Protección de Datos Personales y definir el consentimiento y la finalidad de uso (promociones, fidelización), como exige la Ley N.º 29733 (2011). (c) Aplicar umbrales mínimos de agregación en los indicadores por local o distrito, para que en un local pequeño no se pueda identificar a un cliente o trabajador concreto. (d) Definir periodos de retención y el derecho de cancelación de los datos de clientes y de trabajadores que dejan la empresa. (e) Auditar sesgos en los indicadores: comparar locales por ventas por vendedor sin considerar aforo, zona u horario podría perjudicar a

los de menor demanda al repartir turnos o bonos. (f) Pasar de una sola instancia a roles por local (un administrador de local no debería ver los salarios de otros locales) y a respaldos replicados en varias ubicaciones. Más allá de cumplir la ley, el deber con el cliente y el trabajador también implica decidir qué datos no recolectar.

1.9. Repositorio de código y trazabilidad del proyecto

Todos los artefactos están en el repositorio Git Grupo3_CIIN1021P_EF_REPO, organizado por bloque temático: automatización, seguridad, NoSQL, Data Warehouse/ETL, BI y Big Data. El README.md explica la estructura, el orden de ejecución, las herramientas (todas gratuitas o de código abierto) y los supuestos.

Link: https://github.com/lizandro-marin18/Grupo3_CIIN1021P_EF_REPO.git

Tabla 27

Estructura del repositorio por bloque temático

Carpeta	Contenido principal
00_Datos	Generador de datos, CSV crudos y externos, simulación de la carga y CSV exportados
01_Automatizacion	Scripts T-SQL: esquema, funciones, procedimientos, triggers, pruebas, carga y exportación
02_Seguridad	Roles, enmascaramiento y RLS, índices y plan de ejecución, respaldo y restauración
03_NoSQL_MongoDB	CRUD y agregaciones sobre CatalogoMenu y Resenas
04_DataWarehouse_ETL	ETL en pandas, DDL del DW, CSV del modelo estrella y logs de ejecución
05_BI_PowerBI	Medidas DAX, gobernanza de KPIs, consultas OLAP, resultados, gráficos y dashboard.
06_BigData_Spark	Cuaderno PySpark ejecutado, benchmark de tiempos y gráficos
07_Documentacion	Informe final

Nota. Elaboración propia.

La matriz de trazabilidad enlaza, fila por fila, el logro del curso con el tema del sílabo, el paso de la práctica de campo, la sección del informe y el criterio de rúbrica (Tabla 28).

Tabla 28

Matriz de trazabilidad del proyecto integrador

Logro del curso	Tema	Paso	Secc.	Criterio de rúbrica	EP / EF (pts)
Automatizar procesos de BD con T-SQL	T2	2	1.2	Automatización y justificación de diseño	EF 2; EP 4
Asegurar la BD según la Ley 29733	T3	3	1.3	Seguridad, normativa y consecuencias	EF 2
Integrar SQL y NoSQL	T4	4	1.4	Defensa de decisiones técnicas	EF 6
Diseñar el DW y el ETL	T5	5	1.5	Data Warehouse, ETL y Kimball vs Inmon	EF 2
Construir dashboard, KPIs y OLAP	T6	6	1.6	Claridad y comunicación con datos	EF 6
Procesar Big Data con Spark	T7	7	1.7	Claridad y comunicación con datos	EF 6
Actuar con responsabilidad ética	T1	8	1.8	Argumentación ética	EF 2
Relevar, planificar y gestionar riesgos	T1– T2	1 y 8	1.1, 1.10	Reto, planificación, viabilidad y riesgos	EP 4 + 4 + 4 + 4

Nota. Elaboración propia.

1.10. Planificación, cronograma y ruta crítica

La ruta crítica del proyecto es SQL → CSV → ETL → DW → Dashboard → CSV → Big Data → Informe.

Tabla 29

Cronograma, responsables, ruta crítica y contingencias

Ítem	Actividad (semana)	Responsables	Ruta crítica	Contingencia	Recursos
1	Relevamiento y selección del dataset (S1)	Elmer y Deivys	Inicio	Datos simulados si faltan datos reales	Mockaroo
2	Objetos T-SQL (S2)	Amir y Haziel	Crítica: depende de 1	Revisión cruzada de scripts	SQL Server y SSMS
3	Seguridad y roles (S3)	Elmer y Deivys	Crítica: depende de 2	Respaldos locales diarios	SSMS y scripts de backup
4	NoSQL con MongoDB (S4)	Neda y Amir	Depende de 1	MongoDB Atlas si falla la instalación local	MongoDB y mongosh
5	Validación de viabilidad y EP (S5–S6)	Todos	Crítica: depende de 2 y 3	Reducir volumen a 500 filas	Python y pandas

Ítem	Actividad (semana)	Responsables	Ruta crítica	Contingencia	Recursos
6	Ampliación de datos (más de 5 000 filas) y diagrama E-R (S7)	Haziel y Amir	Crítica: depende de 2	Reducir a 5 000 filas si falla la carga masiva	Python, SQL Server
7	ETL en pandas y Data Warehouse (S7)	Elmer y Deivys	Crítica: depende de 6	Cargar solo a CSV si falla SQL Server	Python, pandas
8	OLAP y dashboard en Power BI (S7–S8)	Neda y Elmer	Crítica: depende de 7	Usar capturas del tablero de referencia	Power BI Desktop
9	Big Data con Spark (S7–S8)	Amir y Haziel	Depende de 7	Ejecutar en Google Colab si falla el entorno local	PySpark, Colab
10	Ética, repositorio, informe y sustentación (S8)	Todos	Entrega final	Ensayo de la exposición (15 minutos)	Git, Word

Nota. Elaboración propia.

1.10.1. Validación de viabilidad

Al inicio, lo más incierto era ejecutar reglas de negocio dentro del motor.

Aislamos `trg_ValidarMontoPedido` y comprobamos que rechaza el monto –10.00

(Figura 4). Con el volumen ampliado, lo más incierto pasó a ser Spark en un equipo de un solo núcleo. Lo aislamos en el cuaderno y corrió sin errores con cuatro fuentes; como evidencias quedaron el log de ejecución (21 celdas) y las métricas de tiempo (0.707 s, 0.913 s y 1.691 s con 40 867, 408 670 y 2 043 350 filas). **Verdicto: viable con ajuste.**

El ajuste concreto fue dejar a Spark como capa complementaria, mantener SQL Server y Power BI como solución operativa y sumar la ejecución en Google Colab como contingencia.

1.10.2. Diagnóstico de riesgos

Tabla 30

Riesgos del proyecto y estrategias de mitigación

Riesgo identificado	Impacto	Estrategia de mitigación	Resultado
Incompatibilidad de formatos en los CSV	Alto	Staging con columnas de texto y validación con TRY_CAST; normalización en el ETL	0 cargas detenidas; 2 755 nombres normalizados
Saturación del servidor por respaldos en horas punta	Medio	FULL el domingo a las 03:00; DIFF diario; LOG cada 15 minutos	Política RPO ≤ 15 min
Pérdida o alteración de la tabla de auditoría	Crítico	DENY al administrador y trigger INSTEAD OF que prohíbe UPDATE y DELETE	Error 51040 verificado en P5
Curva de aprendizaje de NoSQL y Spark	Medio	Dos responsables por tecnología y cuaderno guiado por el de clase	CRUD y 5 análisis en Spark ejecutados
Exposición de datos personales en el entorno analítico	Crítico	Seudonimización, exclusión de datos personales y rol de analista sin acceso	DW sin DNI, correo ni salario
Límites de memoria y CPU en el equipo local	Medio	spark.sql.shuffle.partitions = 8 y memoria del driver en 2 GB; Colab como alternativa	Prueba con 2 043 350 filas exitosa
Indisponibilidad de Power BI o de SQL Server durante la sustentación	Alto	Tablero de referencia, CSV del DW y scripts que reproducen el export	Plan alternativo listo

Nota. Elaboración propia. Se identifican siete riesgos, todos con estrategia de mitigación documentada.

Conclusiones

- **OE1.** Llevamos las reglas de negocio al motor con 4 procedimientos, 5 triggers y 3 funciones. De 12 038 pedidos crudos se aceptaron 11 982 y se rechazaron 18, todos con registro, sin perder trazabilidad.
- **Seguridad.** Seis roles, enmascaramiento, filtro por fila y respaldo FULL + DIFF + LOG cubren los artículos 8, 9, 11, 16 y 17 de la Ley N.º 29733 y siete controles de la norma ISO/IEC 27001:2022.
- **OE2.** El DW en estrella integra SQL Server y un CSV externo en 40 867 líneas de hechos, con 0 llaves nulas y 0 discrepancias. El tablero

responde tres preguntas de negocio con 11 KPIs gobernados y 5 operaciones OLAP.

- **OE3.** Spark procesó cuatro fuentes, incluido el JSON semiestructurado. Con el volumen actual es más lento que SQL Server (0.707 s frente a 0.031 s) y, incluso con 2 043 350 filas, queda 3.90 veces por detrás (1.691 s frente a 0.434 s). Por eso lo usamos como capa de escalabilidad y de análisis semiestructurado, no como reemplazo de SQL Server.
- **OE4 y ética.** El flujo completo se puede reproducir desde el repositorio, y las decisiones de diseño recogen tres dilemas éticos con sus criterios y consecuencias.
- **Limitaciones.** Los datos son simulados, el costo de los platos es una estimación y los tiempos de SQL Server se midieron en un equipo distinto al de pandas, SQLite y Spark, por lo que esa comparación es orientativa.

Referencias

Camuña Rodríguez, J. F. (2014). *Lenguajes de definición y modificación de datos SQL*. IFCT0310. IC Editorial.

Chen, P. P.-S. (1976). The entity-relationship model—Toward a unified view of data.

ACM Transactions on Database Systems, 1(1), 9–36.

<https://doi.org/10.1145/320434.320440>

Colegio de Ingenieros del Perú. (s. f.). *Código de ética del Colegio de Ingenieros del*

Perú. <https://www.cip.org.pe>

Decreto Supremo N.º 016-2024-JUS. Decreto Supremo que aprueba el Reglamento de

la Ley N.º 29733, Ley de Protección de Datos Personales. (2024). Diario Oficial

El Peruano.

Flores Soria, J. (2017). *Finanzas aplicadas a la gestión empresarial: Teoría y práctica*

(4.ª ed.). Centro de Especialización en Contabilidad y Finanzas (CECOF).

- Gómez, M., y Macías, M. (2015). *Introducción a Apache Spark: Para empezar a programar el big data*. eLibro / Biblioteca UPN.
<https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/58454>
- Holmes, D. E. (2018). *Big data: Una breve introducción*. Antoni Bosch Editor.
- Inmon, W. H. (2005). *Building the data warehouse* (4.^a ed.). Wiley.
- International Organization for Standardization y International Electrotechnical Commission. (2022). *ISO/IEC 27001:2022: Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. ISO.
- Kimball, R., y Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3.^a ed.). Wiley.
- Ley N.º 29733. Ley de Protección de Datos Personales. (2011, 3 de julio). Diario Oficial El Peruano.
- McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt y J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- Medina Serrano, S. (2015). *SQL Server 2014: Soluciones prácticas de administración*. eLibro / Biblioteca UPN.
<https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/106467>
- Microsoft. (s. f.-a). *Dynamic data masking*. Microsoft Learn.
<https://learn.microsoft.com/sql/relational-databases/security/dynamic-data-masking>
- Microsoft. (s. f.-b). *Row-level security*. Microsoft Learn.
<https://learn.microsoft.com/sql/relational-databases/security/row-level-security>

Sarasa Cabezuelo, A. (2016). *Introducción a las bases de datos NoSQL usando*

MongoDB. Digitalia / Biblioteca UPN.

<http://digitalia.bibliotecaupn.elogim.com/a/47416>

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen,

J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., y

Stoica, I. (2016). Apache Spark: A unified engine for big data processing.

Communications of the ACM, 59(11), 56–65. <https://doi.org/10.1145/2934664>

II. ANEXOS

Declaración de uso de Inteligencia Artificial

"Declaro que en la elaboración del presente desafío se utilizó inteligencia artificial de manera ética y bajo supervisión humana. La IA fue empleada únicamente para:

Edición lingüística y de estilo (ej.: corrección gramatical, sugerencias de redacción).	(x)
Traducción preliminar de textos.	()
Apoyo en la organización o síntesis de borradores del informe, la presentación o el guion de sustentación, basados en contenidos y evidencias de campo elaborados por la autora/el autor o el equipo.	(x)
Apoyo en el procesamiento o análisis de la información recopilada en la práctica de campo (ej.: tabulación de encuestas/entrevistas, generación de gráficos a partir de datos reales, verificación de cálculos o funciones). Esto debe estar detallado en la parte del entregable correspondiente	(x)
Apoyo en la generación de diagramas, esquemas, mockups o piezas visuales como parte del desarrollo del entregable (especificando parámetros y verificando su fidelidad con la información recopilada). Esto debe estar detallado en la parte del entregable correspondiente	(x)
No se usó Inteligencia Artificial	()

La herramienta de IA utilizada fue: **Claude, de Anthropic (asistente de IA conversacional)**

Todo el contenido generado fue revisado, corregido y validado por el equipo autor, asegurando precisión, integridad académica y fidelidad con los objetivos del trabajo."

Anexo de código fuente

Nota. El código completo está en el repositorio Grupo3_CIIN1021P_EF_REPO. En este anexo se pega el código de cada bloque, con fuente secundaria.

Esquema de la base de datos (01_BaseDatos.sql)

```
REATE DATABASE Cevicheria_DB;
```

```
GO
```

USE Cevicheria_DB;

GO

/* ----- Tablas maestras ----- */

CREATE TABLE dbo.Cliente (

id_cliente INT NOT NULL CONSTRAINT PK_Cliente PRIMARY KEY,

dni CHAR(8) NULL,

nombre VARCHAR(50) NOT NULL,

apellido VARCHAR(80) NOT NULL,

email VARCHAR(100) NULL,

telefono VARCHAR(15) NULL,

distrito VARCHAR(40) NULL,

tipo_cliente VARCHAR(25) NOT NULL CONSTRAINT CK_Cliente_Tipo CHECK (tipo_cliente IN ('Nuevo','Frecuente','Corporativo')),

fecha_registro DATE NOT NULL,

usuario_bd SYSNAME NULL

);

CREATE TABLE dbo.Empleado (

id_empleado INT NOT NULL CONSTRAINT PK_Empleado PRIMARY KEY,

dni_empleado CHAR(8) NOT NULL,

nombre VARCHAR(50) NOT NULL,

apellido VARCHAR(50) NOT NULL,

cargo VARCHAR(30) NOT NULL CONSTRAINT CK_Empleado_Cargo CHECK (cargo IN ('Administrador','Cajero','Mozo','Vendedor','Cocinero','Ayudante','Delivery')),

salario_mensual DECIMAL(10,2) NULL,

usuario_bd SYSNAME NULL

);

CREATE TABLE dbo.Plato (

id_plato INT NOT NULL CONSTRAINT PK_Plato PRIMARY KEY,

nombre_plato VARCHAR(100) NOT NULL CONSTRAINT UQ_Plato_Nombre UNIQUE,

categoria VARCHAR(30) NOT NULL,

precio_base DECIMAL(10,2) NOT NULL CONSTRAINT CK_Plato_Precio CHECK (precio_base > 0),

costo_estimado DECIMAL(10,2) NOT NULL CONSTRAINT CK_Plato_Costo CHECK (costo_estimado >= 0)

);

/* ----- Tablas transaccionales ----- */

CREATE TABLE dbo.Pedido (

id_pedido INT NOT NULL CONSTRAINT PK_Pedido PRIMARY KEY,

fecha_pedido DATE NOT NULL,

hora_pedido TIME(0) NULL,

```

id_cliente INT NOT NULL CONSTRAINT FK_Pedido_Cliente REFERENCES dbo.Cliente(id_cliente), -- 9999 =
cliente de paso

id_empleado INT NOT NULL CONSTRAINT FK_Pedido_Empleado REFERENCES dbo.Empleado(id_empleado),

canal_interno VARCHAR(20) NOT NULL CONSTRAINT CK_Pedido_Canal CHECK (canal_interno IN ('Salón','Para
llevar','Delivery propio')),

metodo_pago VARCHAR(20) NOT NULL CONSTRAINT CK_Pedido_Pago CHECK (metodo_pago IN
('Efectivo','Yape','Plin','Tarjeta')),

monto_total DECIMAL(10,2) NOT NULL, -- regla > 0 la impone el trigger trg_ValidarMontoPedido

fecha_imputada BIT NOT NULL CONSTRAINT DF_Pedido_Imp DEFAULT 0

);

CREATE TABLE dbo.Detalle_Pedido (

id_detalle INT NOT NULL CONSTRAINT PK_Detalle PRIMARY KEY,

id_pedido INT NOT NULL CONSTRAINT FK_Detalle_Pedido REFERENCES dbo.Pedido(id_pedido) ON DELETE
CASCADE,

id_plato INT NOT NULL CONSTRAINT FK_Detalle_Plato REFERENCES dbo.Plato(id_plato),

cantidad INT NOT NULL CONSTRAINT CK_Detalle_Cant CHECK (cantidad > 0),

precio_unitario DECIMAL(10,2) NOT NULL CONSTRAINT CK_Detalle_PU CHECK (precio_unitario > 0),

subtotal DECIMAL(10,2) NOT NULL

);

CREATE TABLE dbo.Asistencia (

id_asistencia INT NOT NULL CONSTRAINT PK_Asistencia PRIMARY KEY,

id_empleado INT NOT NULL CONSTRAINT FK_Asist_Emp REFERENCES dbo.Empleado(id_empleado),

fecha DATE NOT NULL,

hora_ingreso TIME(0) NULL,

horas_trabajadas DECIMAL(5,2) NULL,

tardanza_min INT NOT NULL CONSTRAINT DF_Asist_Tard DEFAULT 0

);

/* ----- Auditoría y errores ( ver trigger trg_ProtegerLogAuditoria) ----- */

CREATE TABLE dbo.Log_Auditoria (

id_log INT IDENTITY(1,1) CONSTRAINT PK_LogAud PRIMARY KEY,

fecha_hora DATETIME2(0) NOT NULL CONSTRAINT DF_LogAud_F DEFAULT SYSDATETIME(),

usuario SYSNAME NOT NULL CONSTRAINT DF_LogAud_U DEFAULT ORIGINAL_LOGIN(),

accion VARCHAR(255) NOT NULL,

tabla_afectada VARCHAR(50) NOT NULL,

valor_anterior VARCHAR(255) NULL,

valor_nuevo VARCHAR(255) NULL

);

CREATE TABLE dbo.Log_Errores (

id_error INT IDENTITY(1,1) CONSTRAINT PK_LogErr PRIMARY KEY,

fecha_hora DATETIME2(0) NOT NULL CONSTRAINT DF_LogErr_F DEFAULT SYSDATETIME(),

```

```
procedimiento SYSNAME    NULL,

numero_error INT         NULL,

mensaje NVARCHAR(4000) NULL,

tabla VARCHAR(50) NULL,

id_registro INT          NULL

);

/* ----- Tablas de paso (staging) para la carga masiva de los CSV crudos ----- */

CREATE TABLE dbo.stg_Pedido (id_pedido VARCHAR(20), fecha_pedido VARCHAR(30), hora_pedido VARCHAR(10),
id_cliente VARCHAR(20),

id_empleado VARCHAR(20), canal_interno VARCHAR(30), metodo_pago VARCHAR(30), monto_total VARCHAR(30));

CREATE TABLE dbo.stg_Detalle (id_detalle VARCHAR(20), id_pedido VARCHAR(20), id_plato VARCHAR(20), cantidad
VARCHAR(20),

precio_unitario VARCHAR(30), subtotal VARCHAR(30));

CREATE TABLE dbo.stg_Cliente (id_cliente VARCHAR(20), dni VARCHAR(20), nombre VARCHAR(60), apellido
VARCHAR(90), distrito VARCHAR(40),

tipo_cliente VARCHAR(30), fecha_registro VARCHAR(20), email VARCHAR(100), telefono VARCHAR(20));

GO

/* Índices de apoyo a FK y a las consultas críticas (ver 06_Indices_Rendimiento.sql para el análisis antes/después) */

CREATE NONCLUSTERED INDEX IX_Pedido_Cliente ON dbo.Pedido(id_cliente);

CREATE NONCLUSTERED INDEX IX_Detalle_Pedido ON dbo.Detalle_Pedido(id_pedido) INCLUDE (id_plato, cantidad,
subtotal);

GO

Seguridad: roles, enmascaramiento y filtro por fila (06_Roles_Privilegios_RLS.sql)

USE Cevicheria_DB;

GO

/* ----- 1. Roles ----- */

IF DATABASE_PRINCIPAL_ID('Rol_Administrador') IS NULL CREATE ROLE Rol_Administrador;

IF DATABASE_PRINCIPAL_ID('Rol_AnalistaDatos') IS NULL CREATE ROLE Rol_AnalistaDatos;

IF DATABASE_PRINCIPAL_ID('Rol_Auditor') IS NULL CREATE ROLE Rol_Auditor;

IF DATABASE_PRINCIPAL_ID('Rol_Vendedor') IS NULL CREATE ROLE Rol_Vendedor; --
mozo/vendedor/cajero: registra ventas

IF DATABASE_PRINCIPAL_ID('Rol_Cliente') IS NULL CREATE ROLE Rol_Cliente; -- cliente
registrado: ve solo SUS pedidos

IF DATABASE_PRINCIPAL_ID('Rol_Cocina') IS NULL CREATE ROLE Rol_Cocina; -- cocina: ve qué
preparar, sin montos ni clientes

GO

/* ----- 2. Vistas de mínimo privilegio ----- */

CREATE OR ALTER VIEW dbo.vw_DW_Cliente AS
```

```
SELECT id_cliente, CONVERT(VARCHAR(64), HASHBYTES('SHA2_256', CONCAT('LaEsquina2026', dni)),  
2) AS dni_hash,
```

```
    distrito, tipo_cliente, fecha_registro FROM dbo.Cliente;
```

```
GO
```

```
CREATE OR ALTER VIEW dbo.vw_DW_Empleado AS SELECT id_empleado, nombre + ' ' + apellido AS  
nombre_completo, cargo FROM dboEmpleado; -- sin DNI ni salario
```

```
GO
```

```
CREATE OR ALTER VIEW dbo.vw_MisPedidos AS SELECT id_pedido, fecha_pedido, canal_interno,  
metodo_pago, monto_total FROM dbo.Pedido; -- RLS filtra por cliente
```

```
GO
```

```
CREATE OR ALTER VIEW dbo.vw_PedidosCocina AS
```

```
    SELECT p.id_pedido, p.hora_pedido, p.canal_interno, pl.nombre_plato, d.cantidad
```

```
    FROM dbo.Pedido p JOIN dbo.Detalle_Pedido d ON d.id_pedido = p.id_pedido JOIN dbo.Plato pl ON pl.id_plato  
= d.id_plato
```

```
    WHERE p.fecha_pedido = CAST(SYSDATETIME() AS DATE);
```

```
GO
```

```
/* ----- 3. Privilegios ----- */
```

```
-- Administrador: CRUD de negocio; ve la auditoría pero NO puede alterarla (además lo impide el trigger INSTEAD  
OF)
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Pedido      TO Rol_Administrador;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Detalle_Pedido TO Rol_Administrador;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Cliente      TO Rol_Administrador;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON dboEmpleado     TO Rol_Administrador;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Plato        TO Rol_Administrador;
```

```
GRANT EXECUTE ON SCHEMA::dbo TO Rol_Administrador;
```

```
GRANT SELECT ON dbo.Log_Auditoria TO Rol_Administrador; DENY INSERT, UPDATE, DELETE ON  
dbo.Log_Auditoria TO Rol_Administrador;
```

```
GRANT UNMASK TO Rol_Administrador;
```

```
-- Analista de datos: solo lectura de vistas seudonimizadas y hechos; sin acceso a tablas con datos personales
```

```
GRANT SELECT ON dbo.vw_DW_Cliente TO Rol_AnalistaDatos; GRANT SELECT ON  
dbo.vw_DW_Empleado TO Rol_AnalistaDatos;
```

```
GRANT SELECT ON dbo.Pedido TO Rol_AnalistaDatos; GRANT SELECT ON dbo.Detalle_Pedido TO  
Rol_AnalistaDatos; GRANT SELECT ON dbo.Plato TO Rol_AnalistaDatos;
```

```
GRANT SELECT ON dbo.fn_VentasPorCategoria TO Rol_AnalistaDatos;
```

```
DENY SELECT ON dbo.Cliente TO Rol_AnalistaDatos; DENY SELECT ON dboEmpleado TO  
Rol_AnalistaDatos;
```

```
DENY SELECT ON dbo.Log_Auditoria TO Rol_AnalistaDatos; DENY SELECT ON dbo.Log_Errores TO
Rol_AnalistaDatos;

-- Auditor: solo logs (no ve datos del negocio) - segregación de funciones (ISO 27001 A.5.3)

GRANT SELECT ON dbo.Log_Auditoria TO Rol_Auditor; GRANT SELECT ON dbo.Log_Errores TO
Rol_Auditor;

DENY SELECT, INSERT, UPDATE, DELETE ON dbo.Pedido TO Rol_Auditor; DENY SELECT ON
dboEmpleado TO Rol_Auditor; DENY SELECT ON dbo.Cliente TO Rol_Auditor;

-- Vendedor: registra ventas solo mediante el SP; ve carta y datos mínimos del cliente; No ve salarios ni DNI

GRANT EXECUTE ON dbo.sp_IngresarPedido TO Rol_Vendedor; GRANT EXECUTE ON
TYPE::dbo.tvp_LineaPedido TO Rol_Vendedor;

GRANT SELECT ON dbo.Plato TO Rol_Vendedor; GRANT SELECT (id_cliente, nombre, apellido,
tipo_cliente) ON dbo.Cliente TO Rol_Vendedor;

DENY DELETE, UPDATE ON dbo.Pedido TO Rol_Vendedor; DENY SELECT ON dboEmpleado TO
Rol_Vendedor; DENY SELECT ON dbo.Log_Auditoria TO Rol_Vendedor;

-- Cliente: solo su historial (RLS) y la carta

GRANT SELECT ON dbo.vw_MisPedidos TO Rol_Cliente; GRANT SELECT ON dbo.Plato TO Rol_Cliente;

DENY SELECT ON dboEmpleado TO Rol_Cliente; DENY SELECT ON dbo.Log_Auditoria TO
Rol_Cliente; DENY SELECT ON dbo.Cliente TO Rol_Cliente;

-- Cocina: solo la vista de preparación

GRANT SELECT ON dbo.vw_PedidosCocina TO Rol_Cocina; DENY SELECT ON dbo.Pedido TO
Rol_Cocina; DENY SELECT ON dbo.Cliente TO Rol_Cocina;

GO

/* ----- 4. Enmascaramiento dinámico ----- */

-- Cliente.dni NO se enmascara: vw_DW_Cliente calcula el hash sobre el DNI real; su protección es por permisos de
columna (Vendedor) y DENY (Analista, Auditor, Cliente, Cocina).

ALTER TABLE dbo.Cliente ALTER COLUMN email ADD MASKED WITH (FUNCTION = 'email()');
ALTER TABLE dbo.Cliente ALTER COLUMN telefono ADD MASKED WITH (FUNCTION =
'partial(0,"XXXXX",4)');

ALTER TABLE dboEmpleado ALTER COLUMN dni_empleado ADD MASKED WITH (FUNCTION =
'partial(0,"XXXX",4)');

ALTER TABLE dboEmpleado ALTER COLUMN salario_mensual ADD MASKED WITH (FUNCTION =
'default()');

GO

/* ----- 5. un cliente solo ve sus pedidos ----- */

CREATE SCHEMA seg AUTHORIZATION dbo;

GO

CREATE OR ALTER FUNCTION seg.fn_FiltroPedidoCliente (@id_cliente INT)

RETURNS TABLE WITH SCHEMABINDING
```

AS RETURN

```
SELECT 1 AS permitido
```

```
WHERE IS_MEMBER('Rol_Cliente') = 0 OR IS_MEMBER('db_owner') = 1           -- los demás roles no  
se filtran
```

```
OR @id_cliente = (SELECT c.id_cliente FROM dbo.Cliente c WHERE c.usuario_bd = USER_NAME());
```

GO

```
CREATE SECURITY POLICY seg.PoliticaPedidoCliente ADD FILTER PREDICATE  
seg.fn_FiltroPedidoCliente(id_cliente) ON dbo.Pedido WITH (STATE = ON);
```

GO

```
/* ----- 6. Usuarios de prueba (sin login) y verificación ----- */
```

```
CREATE USER u_admin WITHOUT LOGIN; ALTER ROLE Rol_Administrador ADD MEMBER u_admin;
```

```
CREATE USER u_analista WITHOUT LOGIN; ALTER ROLE Rol_AnalistaDatos ADD MEMBER u_analista;
```

```
CREATE USER u_auditor WITHOUT LOGIN; ALTER ROLE Rol_Auditor ADD MEMBER u_auditor;
```

```
CREATE USER u_vendedor WITHOUT LOGIN; ALTER ROLE Rol_Vendedor ADD MEMBER u_vendedor;
```

```
CREATE USER u_cliente1 WITHOUT LOGIN; ALTER ROLE Rol_Cliente ADD MEMBER u_cliente1;
```

```
CREATE USER u_cocina WITHOUT LOGIN; ALTER ROLE Rol_Cocina ADD MEMBER u_cocina;
```

```
UPDATE dbo.Cliente SET usuario_bd = 'u_cliente1' WHERE id_cliente = 1;
```

GO

```
PRINT '--- T1 analista: puede leer vista seudonimizada; NO puede leer Empleado (salarios);'
```

```
EXECUTE AS USER = 'u_analista'; SELECT TOP 3 * FROM dbo.vw_DW_Cliente;
```

```
BEGIN TRY SELECT TOP 1 salario_mensual FROM dbo.Empleado; END TRY BEGIN CATCH SELECT  
ERROR_MESSAGE() AS denegado; END CATCH REVERT;
```

```
PRINT '--- T2 vendedor: no puede borrar pedidos ni leer auditoría';
```

```
EXECUTE AS USER = 'u_vendedor'; BEGIN TRY DELETE FROM dbo.Pedido WHERE id_pedido = 1; END TRY  
BEGIN CATCH SELECT ERROR_MESSAGE() AS denegado; END CATCH REVERT;
```

```
PRINT '--- T3 cliente 1 solo ve sus pedidos (RLS);'
```

```
EXECUTE AS USER = 'u_cliente1'; SELECT COUNT(*) AS mis_pedidos, MIN(id_pedido) mn FROM  
dbo.vw_MisPedidos; REVERT;
```

```
PRINT '--- T4 auditor ve logs pero no ventas';
```

```
EXECUTE AS USER = 'u_auditor'; SELECT TOP 3 * FROM dbo.Log_Auditoria; BEGIN TRY SELECT TOP 1 *  
FROM dbo.Pedido; END TRY BEGIN CATCH SELECT ERROR_MESSAGE() AS denegado; END CATCH  
REVERT;
```

```
PRINT '--- T5 enmascaramiento: vendedor/admin leen Cliente con DNI ofuscado o completo según UNMASK';
```

```
EXECUTE AS USER = 'u_admin'; SELECT TOP 2 id_cliente, dni, email FROM dbo.Cliente; REVERT; -- email  
visible por UNMASK del administrador
```

GO

Índices, respaldo y restauración (08_Backup_Restore.sql)

```
USE master;

GO

ALTER DATABASE Cevicheria_DB SET RECOVERY FULL;           -- necesario para respaldos de log

GO

BACKUP DATABASE Cevicheria_DB TO DISK = 'C:\Backups\Cevicheria_DB_FULL.bak'

WITH INIT, FORMAT, CHECKSUM, COMPRESSION, NAME = 'Full semanal', STATS = 10;

BACKUP DATABASE Cevicheria_DB TO DISK = 'C:\Backups\Cevicheria_DB_DIFF.bak'

WITH DIFFERENTIAL, INIT, CHECKSUM, COMPRESSION, NAME = 'Diferencial diario', STATS = 10;

BACKUP LOG Cevicheria_DB TO DISK = 'C:\Backups\Cevicheria_DB_LOG.trn' WITH INIT, CHECKSUM,
COMPRESSION, NAME = 'Log 15 min';

GO

/* Verificación de integridad de los archivos (no restaura, comprueba legibilidad y checksum) */

RESTORE VERIFYONLY FROM DISK = 'C:\Backups\Cevicheria_DB_FULL.bak' WITH CHECKSUM;

RESTORE VERIFYONLY FROM DISK = 'C:\Backups\Cevicheria_DB_DIFF.bak' WITH CHECKSUM;

GO

/* ----- Prueba de restauración en una base de PRUEBA (no toca producción) ----- */

RESTORE FILELISTONLY FROM DISK = 'C:\Backups\Cevicheria_DB_FULL.bak';    -- obtener nombres
lógicos de archivos

RESTORE DATABASE Cevicheria_DB_TEST FROM DISK = 'C:\Backups\Cevicheria_DB_FULL.bak'

WITH NORECOVERY, REPLACE,

    MOVE 'Cevicheria_DB' TO 'C:\Backups\Test\Cevicheria_DB_TEST.mdf',

    MOVE 'Cevicheria_DB_log' TO 'C:\Backups\Test\Cevicheria_DB_TEST.ldf';

RESTORE DATABASE Cevicheria_DB_TEST FROM DISK = 'C:\Backups\Cevicheria_DB_DIFF.bak' WITH
NORECOVERY;

RESTORE LOG Cevicheria_DB_TEST FROM DISK = 'C:\Backups\Cevicheria_DB_LOG.trn' WITH
RECOVERY;

GO

SELECT 'Original' origen, COUNT(*) filas FROM Cevicheria_DB.dbo.Pedido UNION ALL SELECT 'Restaurada',
COUNT(*) FROM Cevicheria_DB_TEST.dbo.Pedido; -- deben coincidir

GO

/* ----- Restauración (producción): expulsa usuarios, restaura FULL + DIFF + LOG ----- */

ALTER DATABASE Cevicheria_DB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE Cevicheria_DB FROM DISK = 'C:\Backups\Cevicheria_DB_FULL.bak' WITH
NORECOVERY, REPLACE;
```

```
RESTORE DATABASE Cevicheria_DB FROM DISK = 'C:\Backups\Cevicheria_DB_DIFF.bak' WITH  
NORECOVERY;  
  
RESTORE LOG Cevicheria_DB FROM DISK = 'C:\Backups\Cevicheria_DB_LOG.trn' WITH RECOVERY;  
  
ALTER DATABASE Cevicheria_DB SET MULTI_USER;  
  
GO
```

MongoDB: CRUD

```
db.createCollection("CatalogoMenu")  
  
// CREATE  
  
db.CatalogoMenu.insertMany([  
  { plato_principal: "Ceviche Clásico", precio_base: 28.00, ingredientes_extra: ["Chifles", "Camote extra"],  
    alergenos: ["pescado"],  
    resenas_clientes: [{ usuario: "juanito99", puntuacion: 5, comentario: "El pescado súper fresco." } ] },  
  { plato_principal: "Leche de Tigre Especial", precio_base: 24.00, temporada: "verano", alergenos: ["pescado",  
    "mariscos" ] },  
  { plato_principal: "Ceviche Vegetariano", precio_base: 24.00, vegetariano: true, ingredientes_extra: ["Palta" ] })  
  
// READ  
  
db.CatalogoMenu.find({ ingredientes_extra: "Chifles" })  
  
db.CatalogoMenu.find({ alergenos: { $in: ["mariscos" ] }, { plato_principal: 1, precio_base: 1, _id: 0 })  
  
// UPDATE  
  
db.CatalogoMenu.updateOne({ plato_principal: "Ceviche Clásico" }, { $set: { precio_base: 26.00 }, $push: {  
  ingredientes_extra: "Cancha" } })  
  
// DELETE  
  
db.CatalogoMenu.deleteOne({ plato_principal: "Ceviche Vegetariano" })
```

DDL del Data Warehouse

```
CREATE DATABASE DW_Cevicheria;  
  
GO  
  
USE DW_Cevicheria;  
  
GO  
  
/* Re-ejecutable: elimina primero el hecho (tiene las FK) y luego las dimensiones */  
  
DROP TABLE IF EXISTS dbo.Fact_Ventas;  
  
DROP TABLE IF EXISTS dbo.Dim_Fecha, dbo.Dim_Cliente, dbo.Dim_Plato, dbo.Dim_Empleado,  
dbo.Dim_MetodoPago, dbo.Dim_Canal, dbo.Dim_Turno;  
  
GO  
  
CREATE TABLE dbo.Dim_Fecha (sk_fecha INT NOT NULL PRIMARY KEY, fecha DATE NOT NULL, anio  
SMALLINT, trimestre TINYINT, mes_num TINYINT, mes_nombre VARCHAR(12),
```

```
anio_mes CHAR(7), semana_iso TINYINT, dia_mes TINYINT, dia_semana_num TINYINT, dia_semana
VARCHAR(10), es_fin_semana BIT, es_feriado BIT, nombre_feriado VARCHAR(40) NULL);

CREATE TABLE dbo.Dim_Cliente (sk_cliente INT NOT NULL PRIMARY KEY, id_cliente_nk INT NOT NULL,
cliente_seudonimo VARCHAR(12) NOT NULL, dni_hash CHAR(64) NULL,

    distrito VARCHAR(40), tipo_cliente VARCHAR(25), fecha_registro DATE, antiguedad_meses INT,
es_cliente_identificado BIT);

CREATE TABLE dbo.Dim_Plato (sk_plato INT NOT NULL PRIMARY KEY, id_plato_nk INT NOT NULL,
nombre_plato VARCHAR(100), categoria VARCHAR(30), precio_base DECIMAL(10,2),

    costo_estimado DECIMAL(10,2), margen_unitario DECIMAL(10,2), margen_pct DECIMAL(5,1), rango_precio
VARCHAR(20));

CREATE TABLE dbo.Dim_Empleado (sk_empleado INT NOT NULL PRIMARY KEY, id_empleado_nk INT NOT
NULL, nombre_completo VARCHAR(110), cargo VARCHAR(30)); -- sin DNI ni salario

CREATE TABLE dbo.Dim_MetodoPago (sk_pago INT NOT NULL PRIMARY KEY, metodo_pago
VARCHAR(30), tipo_pago VARCHAR(20));

CREATE TABLE dbo.Dim_Canal (sk_canal INT NOT NULL PRIMARY KEY, canal VARCHAR(20), tipo_canal
VARCHAR(20), origen_datos VARCHAR(15));

CREATE TABLE dbo.Dim_Turno (sk_turno INT NOT NULL PRIMARY KEY, turno VARCHAR(20));

CREATE TABLE dbo.Fact_Ventas (

    sk_venta INT NOT NULL PRIMARY KEY,

    sk_fecha INT NOT NULL REFERENCES dbo.Dim_Fecha(sk_fecha), sk_cliente INT NOT NULL
REFERENCES dbo.Dim_Cliente(sk_cliente),

    sk_plato INT NOT NULL REFERENCES dbo.Dim_Plato(sk_plato), sk_empleado INT NOT NULL
REFERENCES dbo.Dim_Empleado(sk_empleado),

    sk_pago INT NOT NULL REFERENCES dbo.Dim_MetodoPago(sk_pago), sk_canal INT NOT NULL
REFERENCES dbo.Dim_Canal(sk_canal), sk_turno INT NOT NULL REFERENCES dbo.Dim_Turno(sk_turno),

    nro_pedido VARCHAR(20) NOT NULL, origen VARCHAR(15) NOT NULL,

    cantidad INT NOT NULL, precio_unitario DECIMAL(10,2) NOT NULL, importe_bruto DECIMAL(10,2) NOT
NULL, comision_plataforma DECIMAL(10,2) NOT NULL,

    importe_netto DECIMAL(10,2) NOT NULL, costo_total DECIMAL(10,2) NOT NULL, tiempo_entrega_min INT
NULL, calificacion DECIMAL(2,1) NULL, margen_bruto DECIMAL(10,2) NOT NULL);

GO

CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Fact_Ventas ON dbo.Fact_Ventas (sk_fecha,
sk_plato, sk_canal, sk_cliente, importe_netto, importe_bruto, costo_total, cantidad); -- acelera agregaciones OLAP

GO

/* ----- Carga: dimensiones primero, hecho al final (orden por las FK) ----- */

DECLARE @r VARCHAR(200) =
'C:\Grupo3_CIIIN1021P_EF_ENTREGA\Grupo3_CIIIN1021P_EF_REPO\04_DataWarehouse_ETL\DW_csv\';
```

```
DECLARE @rt VARCHAR(10) = '0x0a';

DECLARE @o NVARCHAR(300) = N' WITH (FORMAT="CSV", FIRSTROW=2, FIELDTERMINATOR="," ,
ROWTERMINATOR="'" + @rt + N"'", CODEPAGE="65001", KEEPNULLS, TABLOCK);';

EXEC(N'BULK INSERT dbo.Dim_Fecha FROM '" + @r + 'dim_fecha.csv"' + @o);
EXEC(N'BULK INSERT dbo.Dim_Cliente FROM '" + @r + 'dim_cliente.csv"' + @o);
EXEC(N'BULK INSERT dbo.Dim_Plato FROM '" + @r + 'dim_plato.csv"' + @o);
EXEC(N'BULK INSERT dbo.Dim_Empleado FROM '" + @r + 'dim_empleado.csv"' + @o);
EXEC(N'BULK INSERT dbo.Dim_MetodoPago FROM '" + @r + 'dim_metodo_pago.csv"' + @o);
EXEC(N'BULK INSERT dbo.Dim_Canal FROM '" + @r + 'dim_canal.csv"' + @o);
EXEC(N'BULK INSERT dbo.Dim_Turno FROM '" + @r + 'dim_turno.csv"' + @o);
EXEC(N'BULK INSERT dbo.Fact_Ventas FROM '" + @r + 'fact_ventas.csv"' + @o);

GO
```